Command Prompt - nslookup

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\TCS>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a             Displays current ARP entries by interrogating the current
                 protocol data.  If inet_addr is specified, the IP and Physical
                 addresses for only the specified computer are displayed.  If
                 more than one network interface uses ARP, entries for each ARP
                 table are displayed.
  -g             Same as -a.
  -v             Displays current ARP entries in verbose mode.  All invalid
                 entries and entries on the loop-back interface will be shown.
  inet_addr      Specifies an internet address.
  -N if_addr     Displays the ARP entries for the network interface specified
                 by if_addr.
  -d             Deletes the host specified by inet_addr. inet_addr may be
                 wildcarded with * to delete all hosts.
  -s             Adds the host and associates the Internet address inet_addr
                 with the Physical address eth_addr.  The Physical address is
                 given as 6 hexadecimal bytes separated by hyphens. The entry
                 is permanent.
  eth_addr       Specifies a physical address.
  if_addr        If present, this specifies the Internet address of the
                 interface whose address translation table should be modified.
                 If not present, the first applicable interface will be used.
Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
  > arp -a                                    .... Displays the arp table.

C:\Users\TCS>hostname
DESKTOP-F4RM5ER
```

```
C:\Users\TCS>hostname
DESKTOP-F4RM5ER

C:\Users\TCS>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::d51f:7aad:f478:5629%7
   IPv4 Address. . . . . . . . . . . : 172.16.11.192
   Subnet Mask . . . . . . . . . . . : 255.255.252.0
   Default Gateway . . . . . . . . . : 172.16.8.1

C:\Users\TCS>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.
```

Command Prompt - nslookup

```
C:\Users\TCS>netstat

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    172.16.11.192:51810    4.213.25.240:https     ESTABLISHED
  TCP    172.16.11.192:51812    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51813    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51814    a23-211-60-142:https   CLOSE_WAIT
  TCP    172.16.11.192:51815    a23-211-60-142:https   CLOSE_WAIT
  TCP    172.16.11.192:51816    a23-211-60-142:https   CLOSE_WAIT
  TCP    172.16.11.192:51817    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51818    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51819    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51820    a23-199-67-57:https    CLOSE_WAIT
  TCP    172.16.11.192:51821    a23-199-67-57:https    CLOSE_WAIT
  TCP    172.16.11.192:52699    123:http               TIME_WAIT
  TCP    172.16.11.192:52700    123:http               TIME_WAIT
  TCP    172.16.11.192:52701    150.171.28.12:https    ESTABLISHED
  TCP    172.16.11.192:52702    a23-202-229-22:https   CLOSE_WAIT
  TCP    172.16.11.192:52703    4.207.44.66:https      TIME_WAIT
  TCP    172.16.11.192:52704    a23-202-229-22:https   ESTABLISHED
  TCP    172.16.11.192:52705    4.213.25.242:https     ESTABLISHED
  TCP    172.16.11.192:52706    123:http               ESTABLISHED

C:\Users\TCS>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
    -g host-list      Loose source route along host-list.
    -h maximum_hops   Maximum number of hops to search for target.
    -i address        Use the specified source address.
    -n                Do not resolve addresses to hostnames.
    -p period         Wait period milliseconds between pings.
    -q num_queries    Number of queries per hop.
    -w timeout        Wait timeout milliseconds for each reply.
    -4                Force using IPv4.
    -6                Force using IPv6.
```

```
C:\Users\TCS>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t              Ping the specified host until stopped.
                    To see statistics and continue - type Control-Break;
                    To stop - type Control-C.
    -a              Resolve addresses to hostnames.
    -n count        Number of echo requests to send.
    -l size         Send buffer size.
    -f              Set Don't Fragment flag in packet (IPv4-only).
    -i TTL          Time To Live.
    -v TOS          Type Of Service (IPv4-only. This setting has been deprecated
                    and has no effect on the type of service field in the IP
                    Header).
    -r count        Record route for count hops (IPv4-only).
    -s count        Timestamp for count hops (IPv4-only).
    -j host-list    Loose source route along host-list (IPv4-only).
    -k host-list    Strict source route along host-list (IPv4-only).
    -w timeout      Timeout in milliseconds to wait for each reply.
    -R              Use routing header to test reverse route also (IPv6-only).
                    Per RFC 5095 the use of this routing header has been
                    deprecated. Some systems may drop echo requests if
                    this header is used.
    -S srcaddr      Source address to use.
    -c compartment  Routing compartment identifier.
    -p              Ping a Hyper-V Network Virtualization provider address.
    -4              Force using IPv4.
    -6              Force using IPv6.
```

Command Prompt - nslookup

```
C:\Users\TCS>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

  -f            Clears the routing tables of all gateway entries.  If this is
                used in conjunction with one of the commands, the tables are
                cleared prior to running the command.

  -p            When used with the ADD command, makes a route persistent across
                boots of the system. By default, routes are not preserved
                when the system is restarted. Ignored for all other commands,
                which always affect the appropriate persistent routes.

  -4            Force using IPv4.

  -6            Force using IPv6.

  command       One of these:
                  PRINT     Prints  a route
                  ADD       Adds    a route
                  DELETE    Deletes a route
                  CHANGE    Modifies an existing route
  destination   Specifies the host.
  MASK          Specifies that the next parameter is the 'netmask' value.
  netmask       Specifies a subnet mask value for this route entry.
                If not specified, it defaults to 255.255.255.255.
  gateway       Specifies gateway.
  interface     the interface number for the specified route.
  METRIC        specifies the metric, ie. cost for the destination.
```

Command Prompt - nslookup

```
All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.

If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Destination & Mask) != Destination.

Examples:

    > route PRINT
    > route PRINT -4
    > route PRINT -6
    > route PRINT 157*          .... Only prints those matching 157*

    > route ADD 157.0.0.0 MASK 255.0.0.0  157.55.80.1 METRIC 3 IF 2
             destination^      ^mask      ^gateway     metric^    ^
                                                       Interface^
      If IF is not given, it tries to find the best interface for a given
      gateway.
    > route ADD 3ffe::/32 3ffe::1

    > route CHANGE 157.0.0.0 MASK 255.0.0.0 157.55.80.5 METRIC 2 IF 2

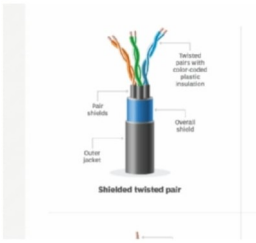
      CHANGE is used to modify gateway and/or metric only.

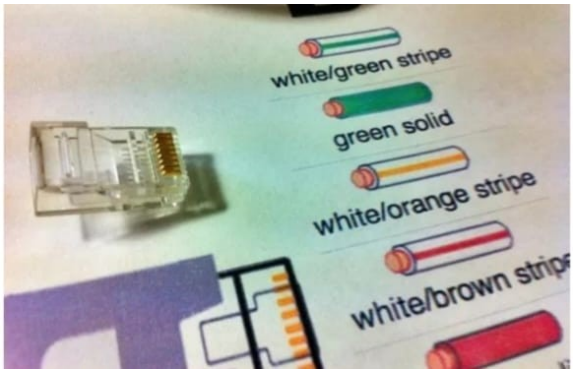
    > route DELETE 157.0.0.0
    > route DELETE 3ffe::/32
```

```
C:\Users\TCS>nslookup www.google.com
Server:  UnKnown
Address:  172.16.8.1

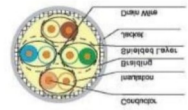
Non-authoritative answer:
Name:    www.google.com
Addresses:  2001:4860:4827:7700::
          2001:4860:482b:7700::
          2001:4860:4828:7700::
          2001:4860:4826:7700::
          2001:4860:482d:7700::
          2001:4860:4829:7700::
          2001:4860:482c:7700::
          2001:4860:482a:7700::
          142.251.156.119
          142.251.153.119
          142.251.157.119
          142.251.152.119
          142.251.151.119
          142.251.154.119
          142.251.155.119
          142.251.150.119


C:\Users\TCS>
```

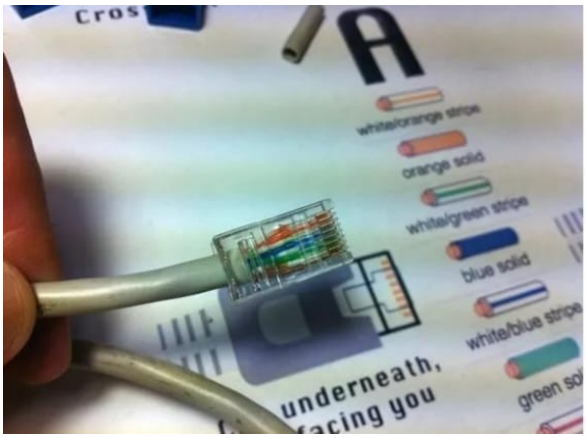




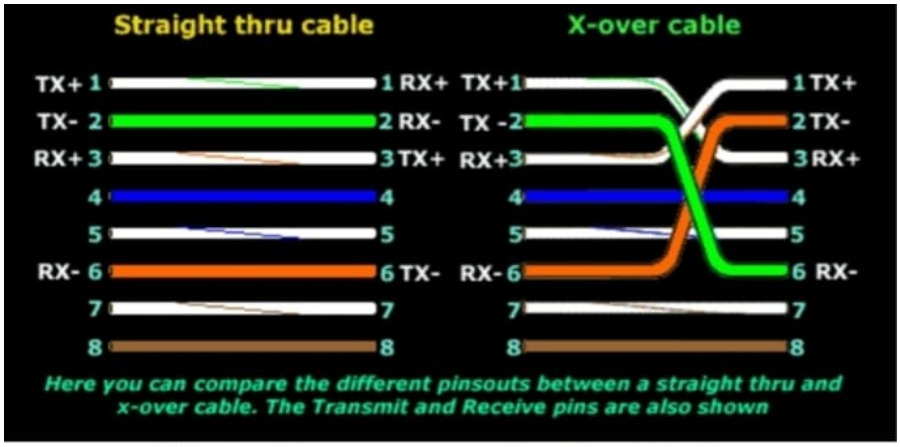




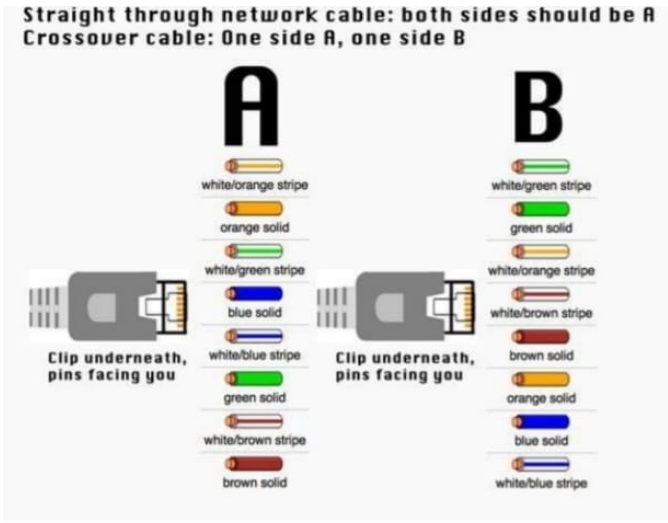




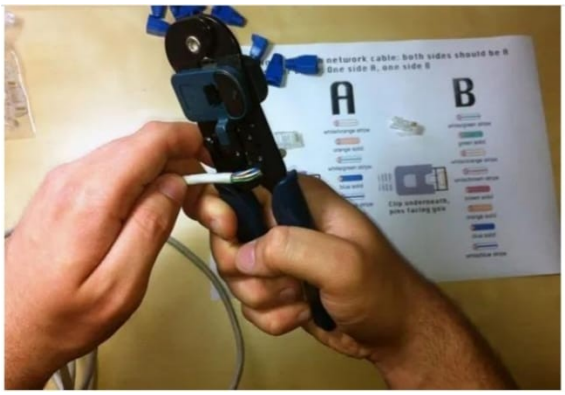


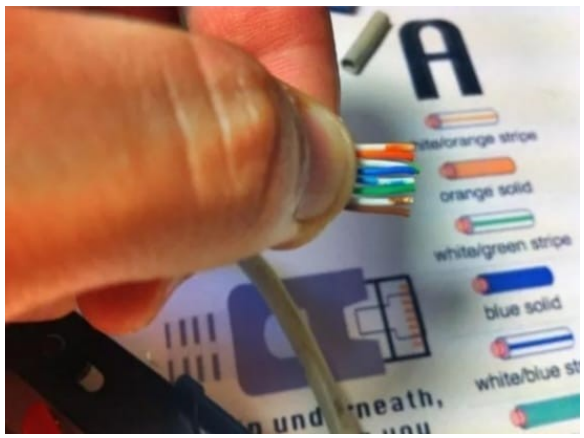


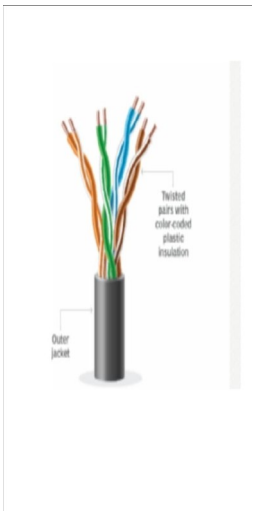
Difference between crossover cable and straight cable

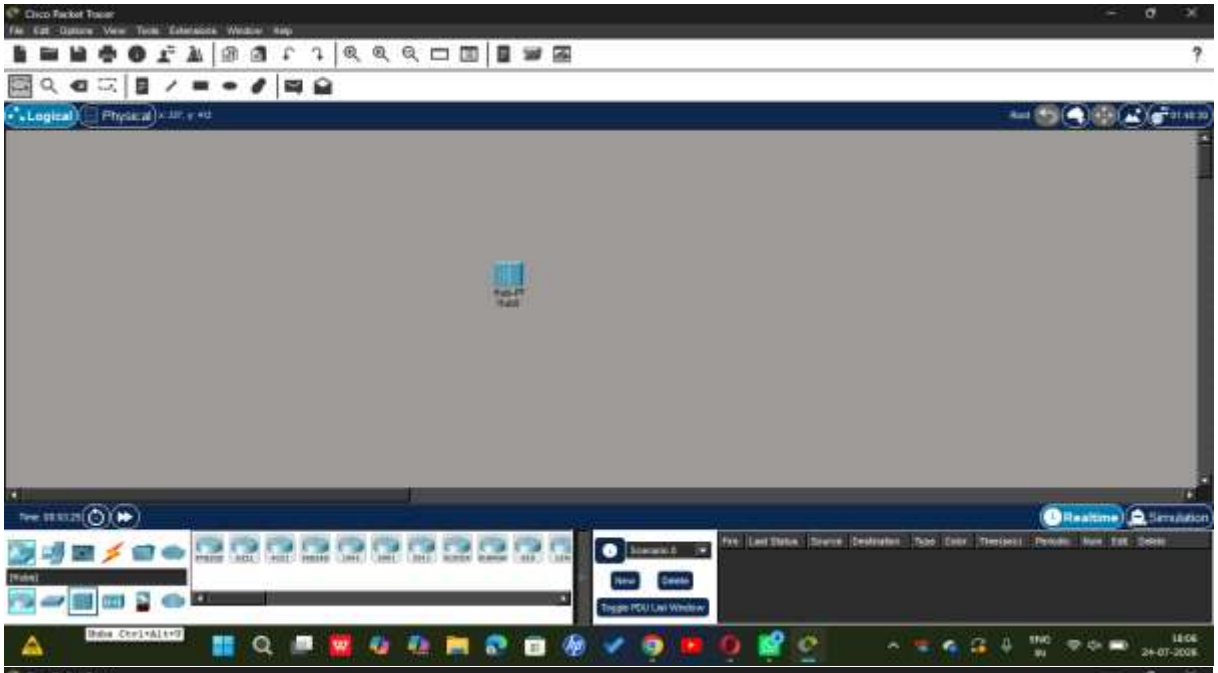


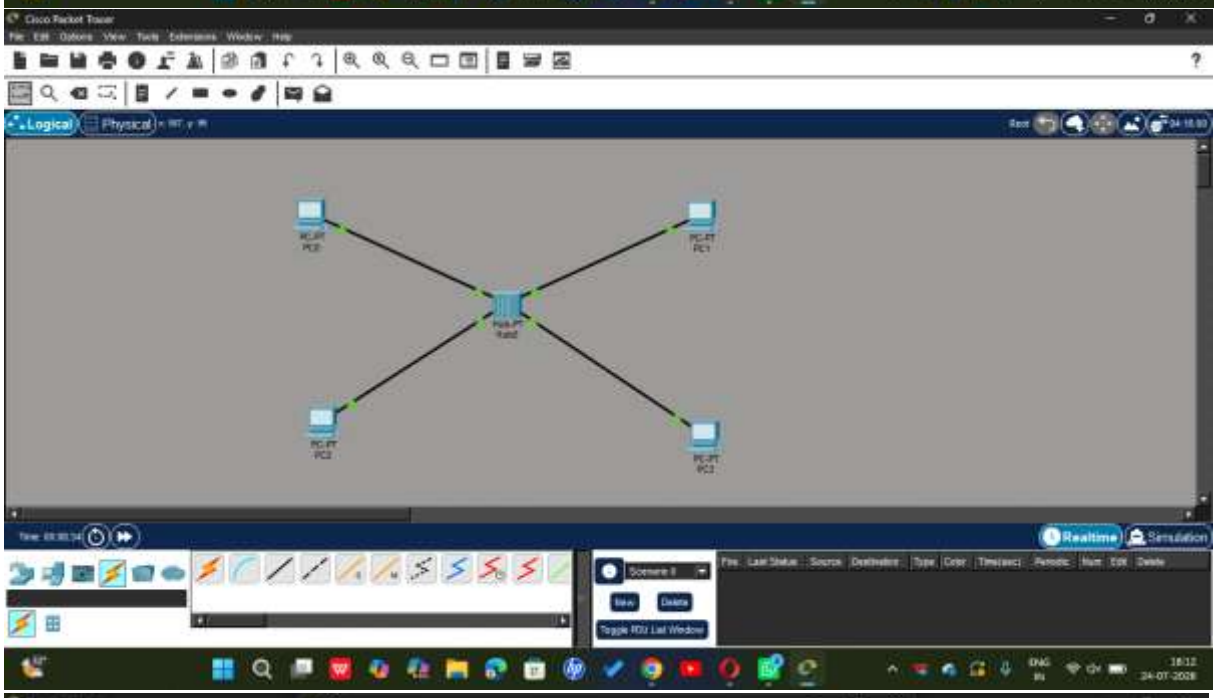


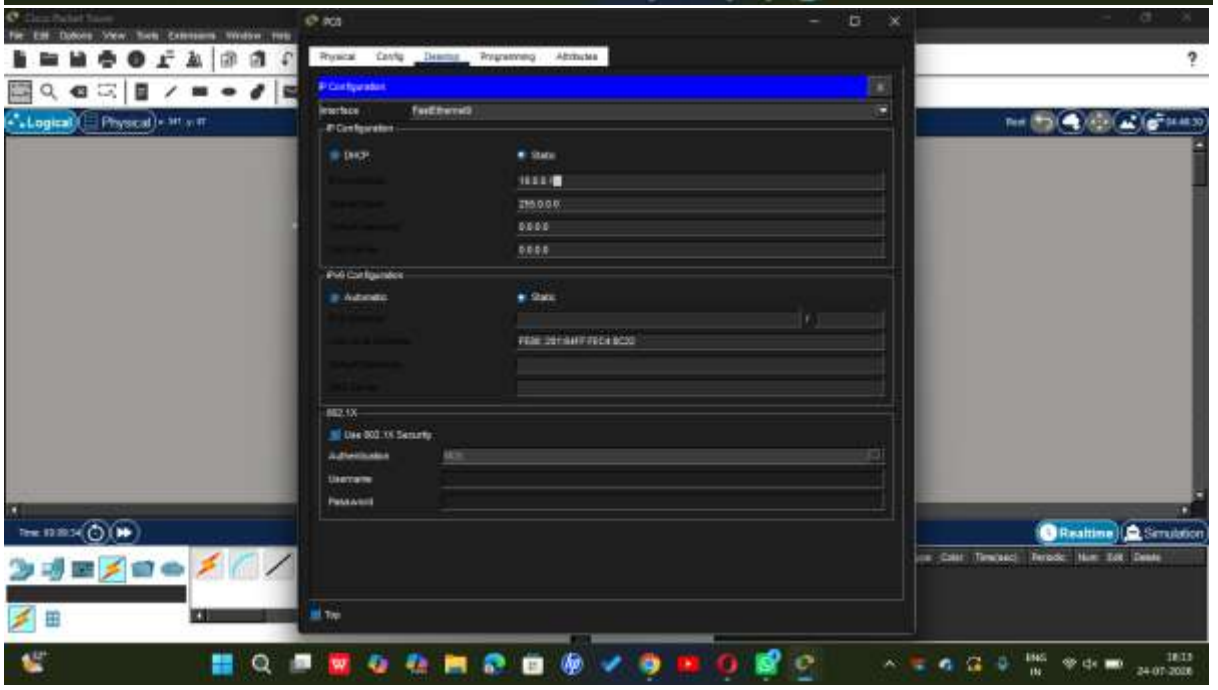


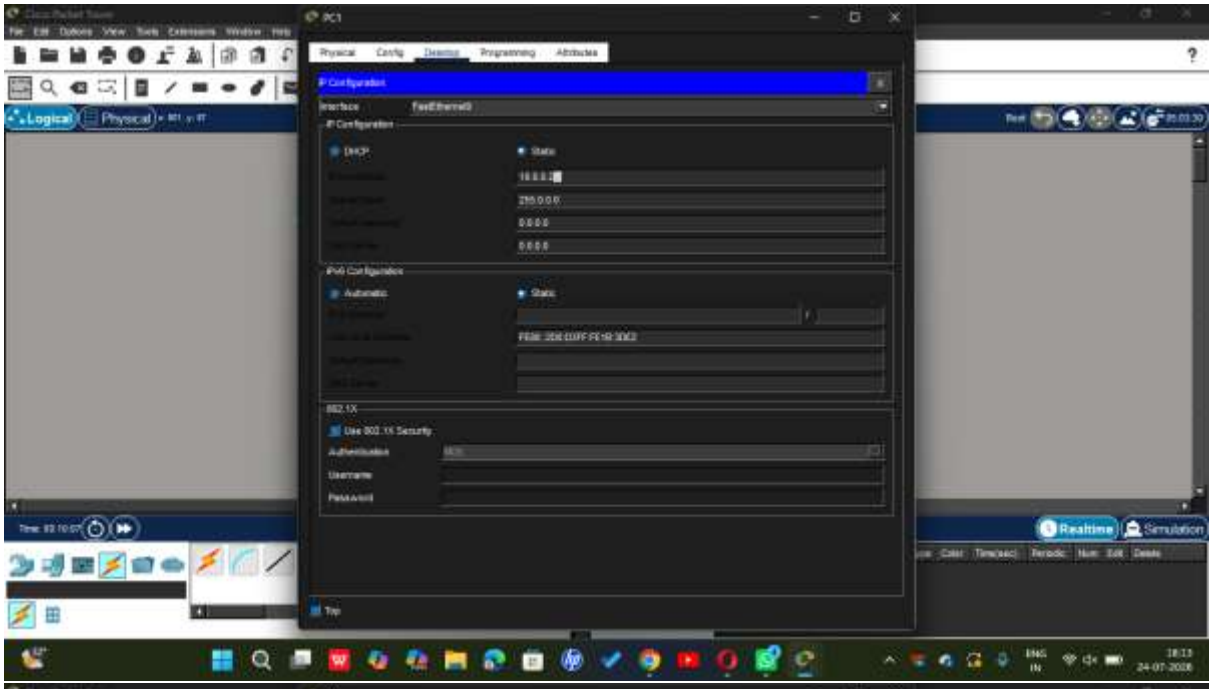


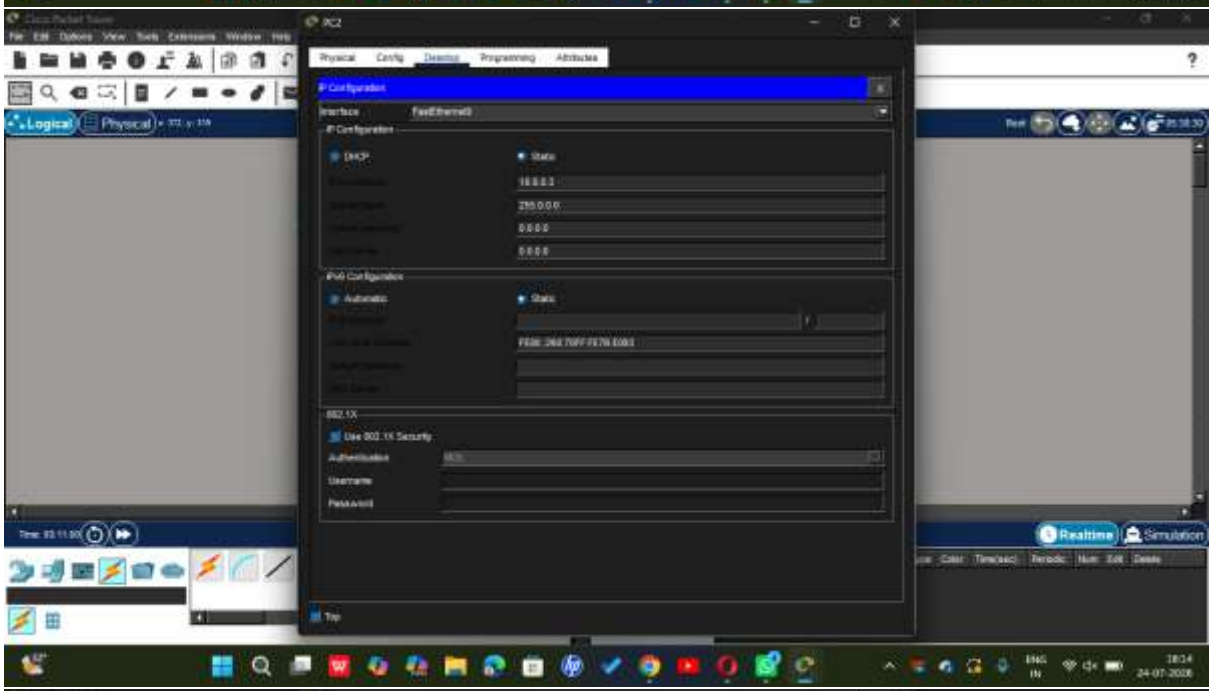


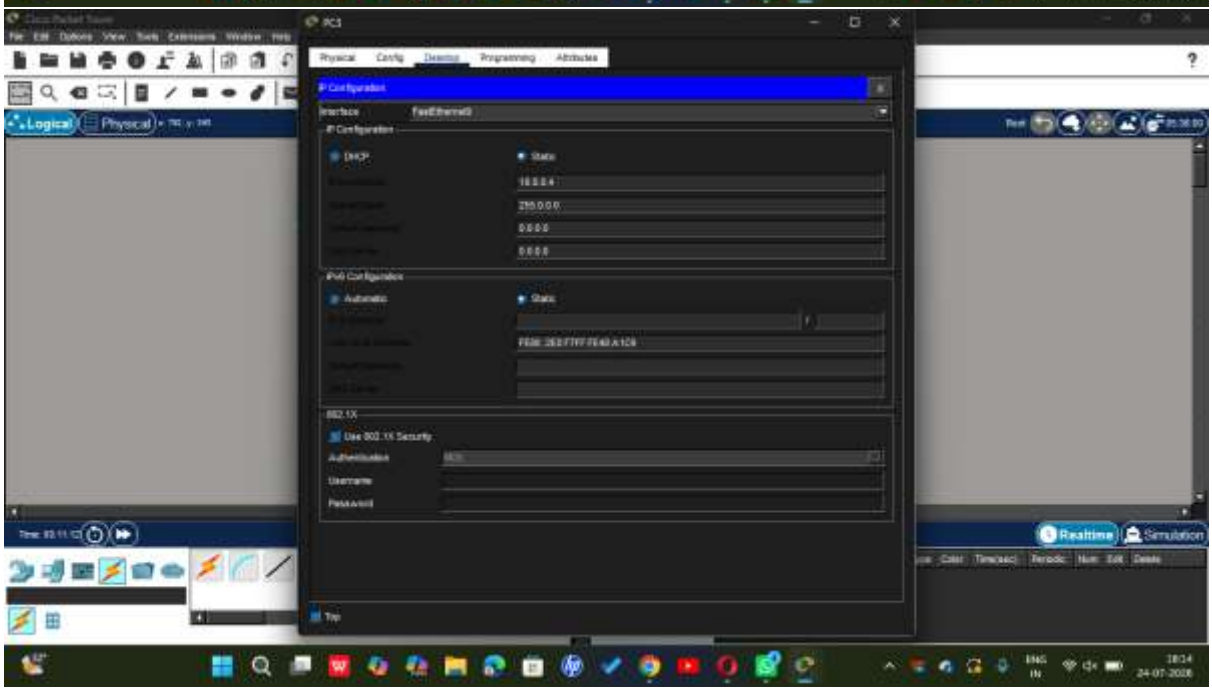


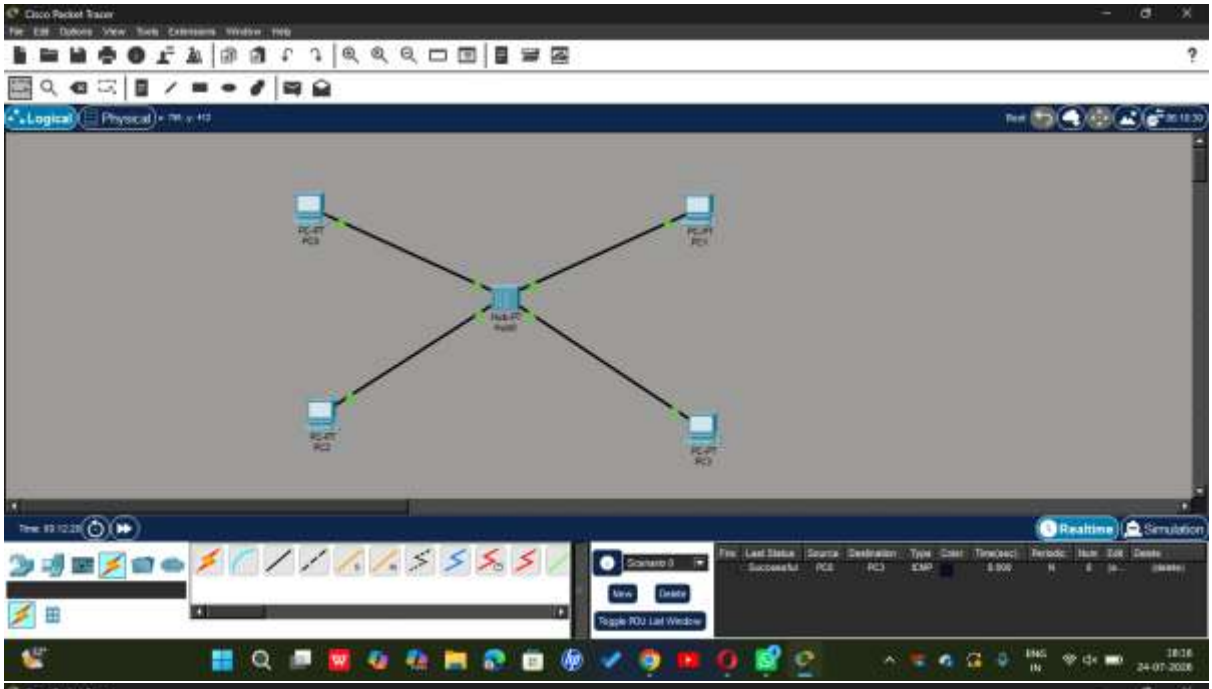


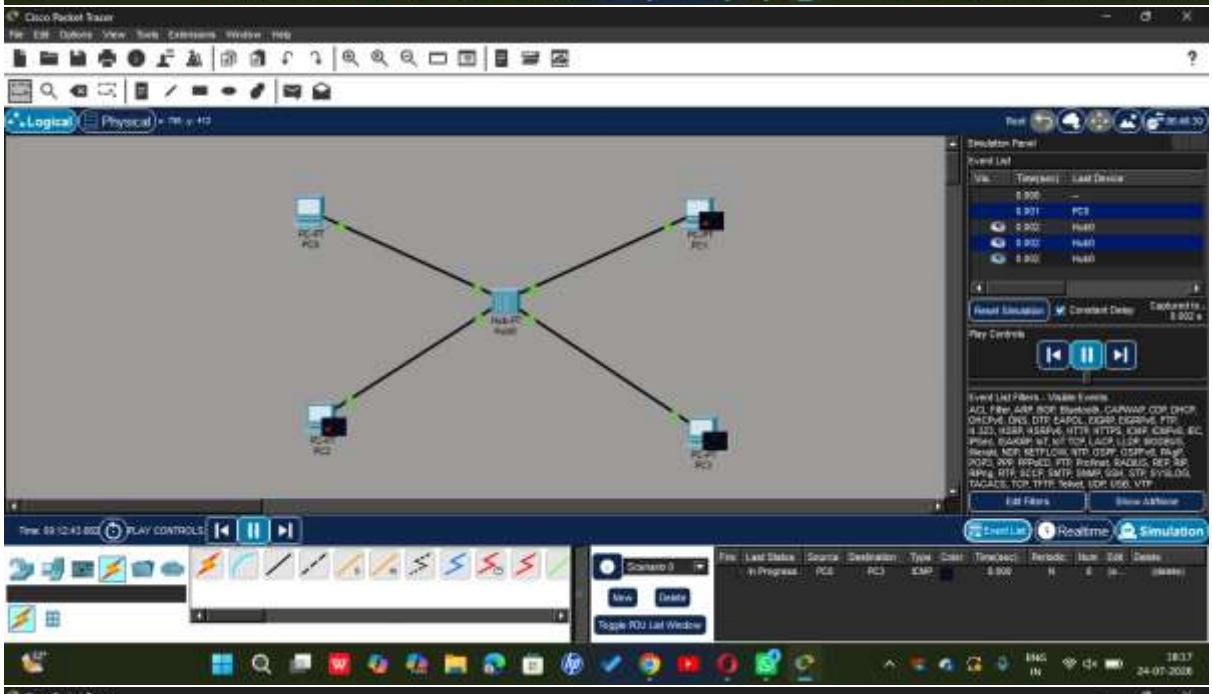


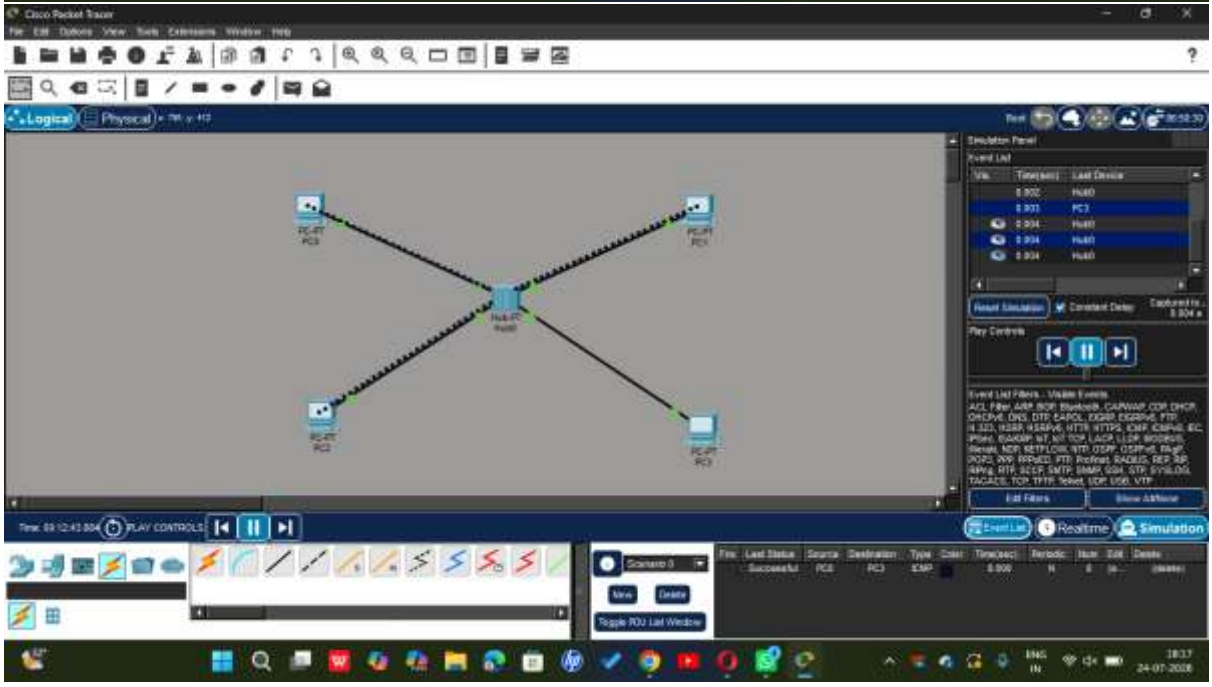

PC1
Physical
Config
Desktop
Programming
Attributes
IP Configuration
Interface
FastEthernet0
DHCP
Static
IPv6 Configuration
Automatic
802.1X
Use 802.1X Security
Authentication
Username
Password
Logical
Physical
Realtime
Simulation
Top

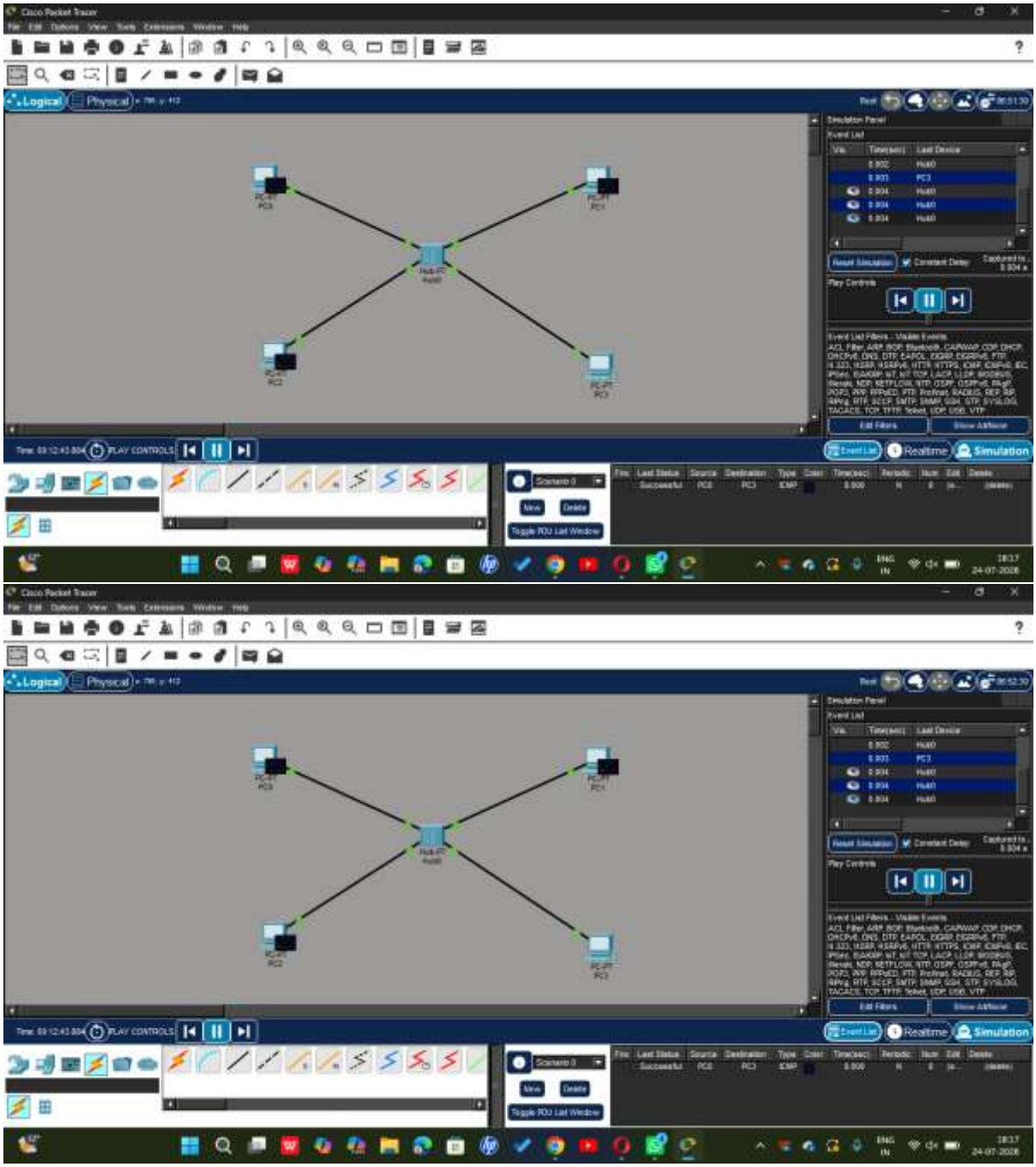

PC2
Physical
Config
Desktop
Programming
Attributes
IP Configuration
Interface
FastEthernet0
DHCP
Static
IPv6 Configuration
Automatic
802.1X
Use 802.1X Security
Authentication
Username
Password
Logical
Physical
Realtime
Simulation
Top

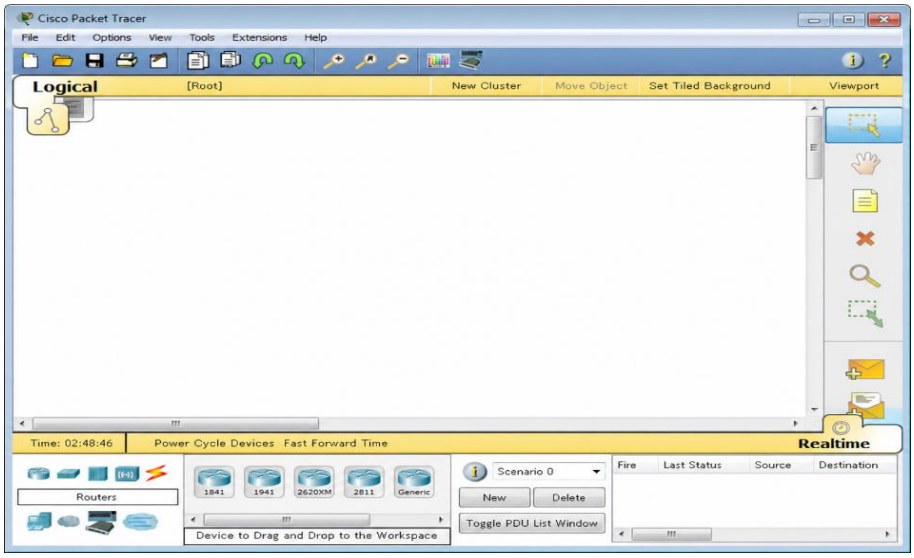

PC3
Physical
Config
Desktop
Programming
Attributes
IP Configuration
Interface
FastEthernet0
DHCP
Static
IPv6 Configuration
Automatic
802.1X
Use 802.1X Security
Authentication
Username
Password
Logical
Physical
Realtime
Simulation
Top



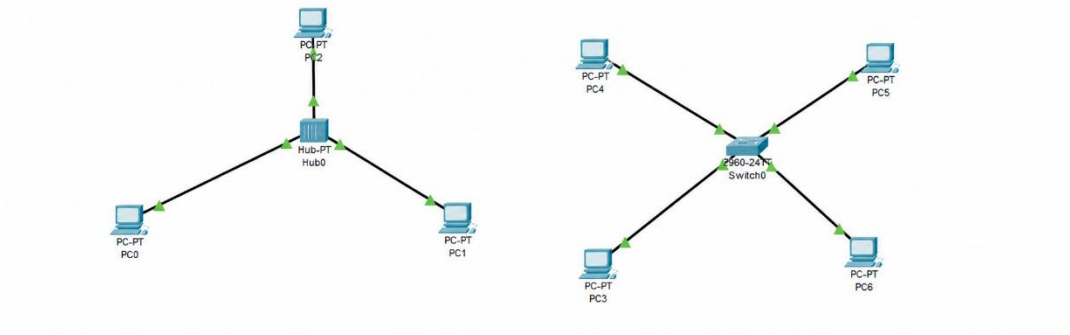


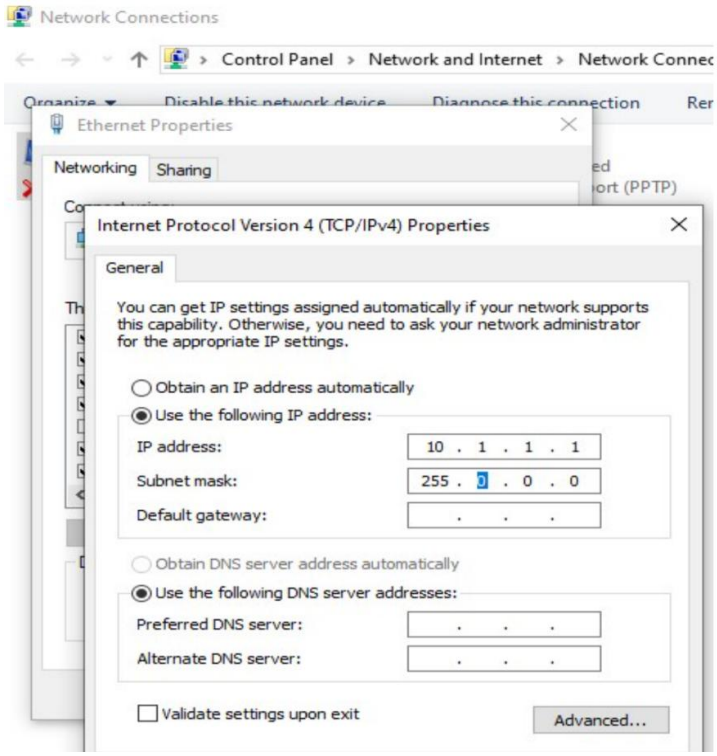


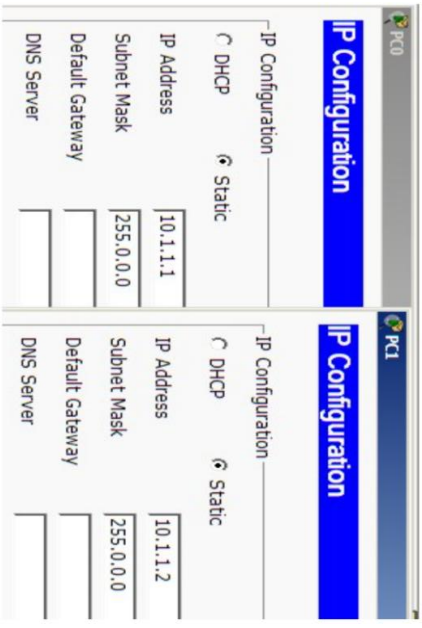


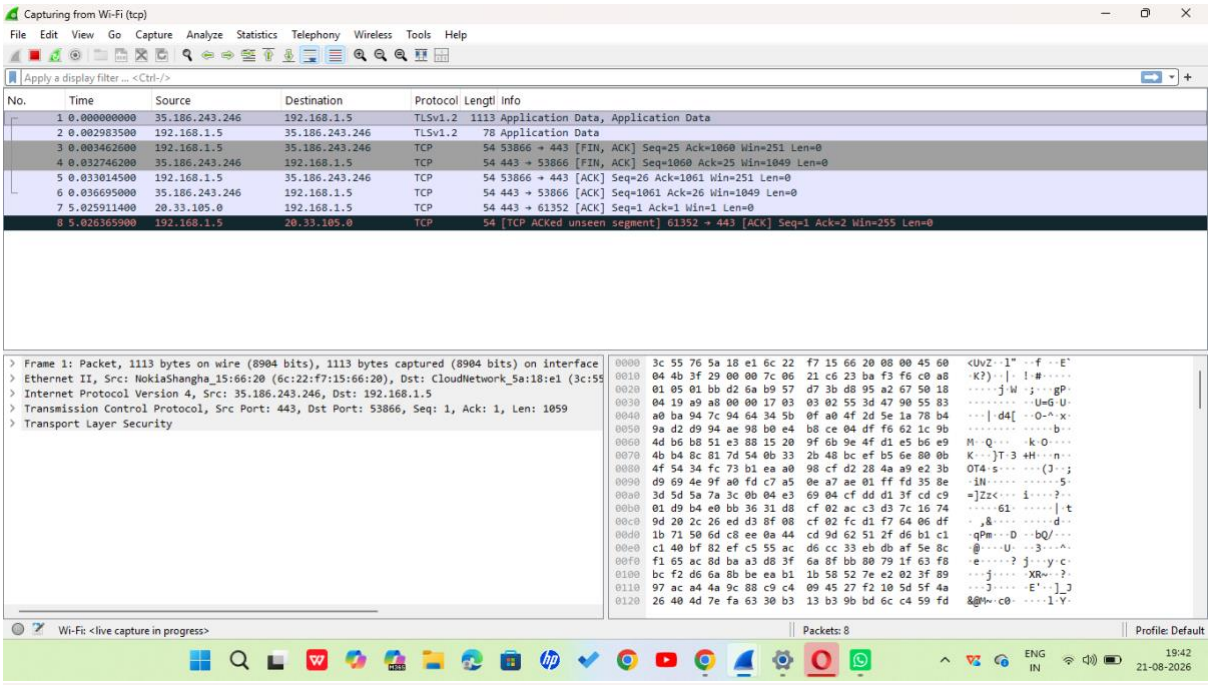

Cisco Packet Tracer
File Edit Options View Tools Extensions Help
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
Time: 02:48:46
Power Cycle Devices
Fast Forward Time
Realtime
Routers
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Device to Drag and Drop to the Workspace

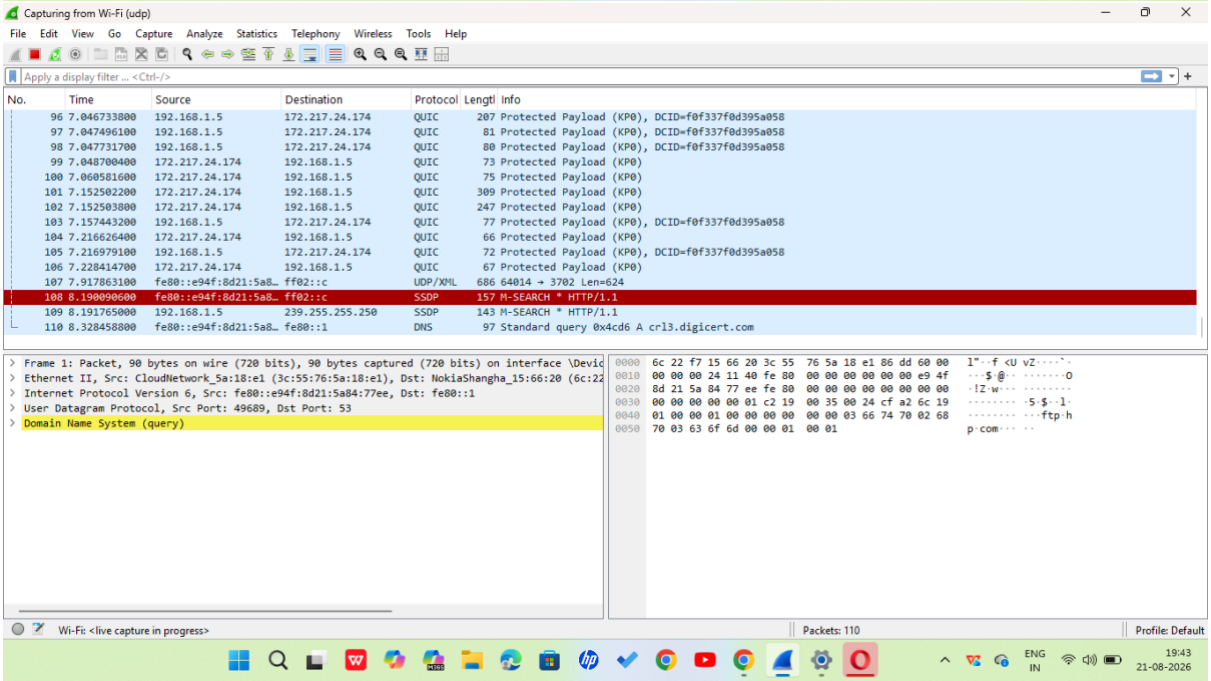

File Edit Options View Tools Extensions Window Help
Logical
Physical

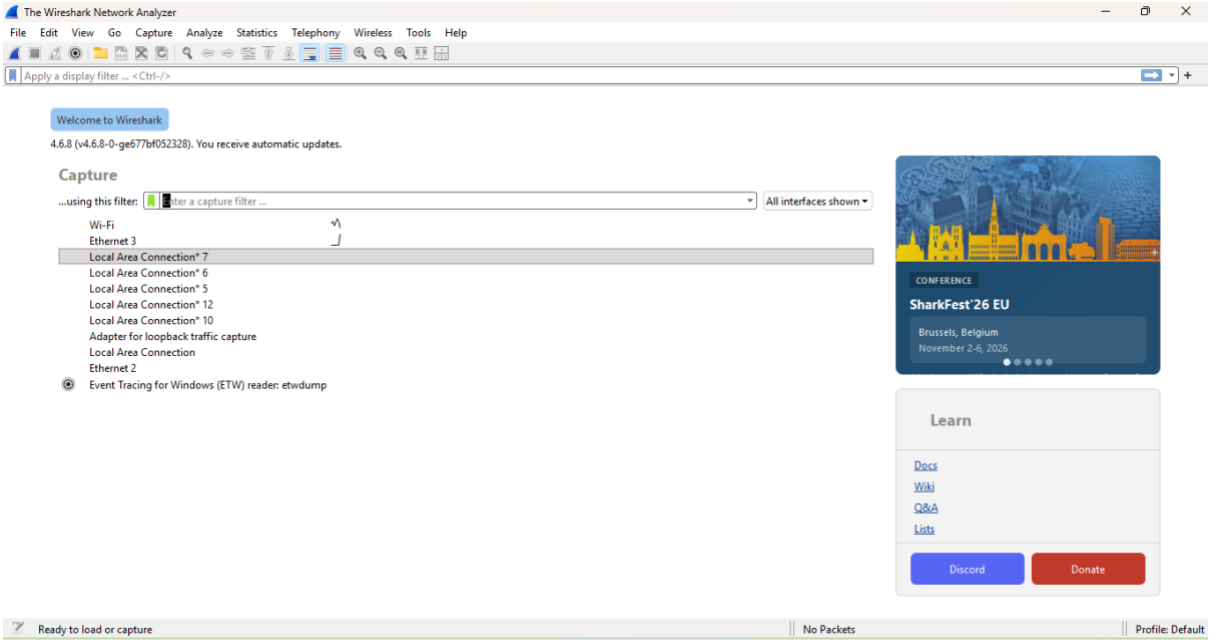

PC-PT
PC2
Hub-PT
Hub0
PC-PT
PC0
PC-PT
PC1
PC-PT
PC4
PC-PT
PC5
2960-24TT
Switch0
PC-PT
PC3
PC-PT
PC6


Network Connections
Control Panel
Network and Internet
Network Connec
Ethernet Properties
Networking
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address:
10 . 1 . 1 . 1
Subnet mask:
255 . 0 . 0 . 0
Default gateway:
Obtain DNS server address automatically
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...


PC0
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.1
255.0.0.0
PC1
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.2
255.0.0.0


Capturing from Wi-Fi (tcp)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Lengt Info
1 0.000000000 35.186.243.246 192.168.1.5 TLSv1.2 1113 Application Data, Application Data
2 0.002983500 192.168.1.5 35.186.243.246 TLSv1.2 78 Application Data
3 0.003462600 192.168.1.5 35.186.243.246 TCP 54 53866 → 443 [FIN, ACK] Seq=25 Ack=1060 Win=251 Len=0
4 0.032746200 35.186.243.246 192.168.1.5 TCP 54 443 → 53866 [FIN, ACK] Seq=1060 Ack=25 Win=1049 Len=0
5 0.033014500 192.168.1.5 35.186.243.246 TCP 54 53866 → 443 [ACK] Seq=26 Ack=1061 Win=251 Len=0
6 0.036695000 35.186.243.246 192.168.1.5 TCP 54 443 → 53866 [ACK] Seq=1061 Ack=26 Win=1049 Len=0
7 5.025911400 20.33.105.0 192.168.1.5 TCP 54 443 → 61352 [ACK] Seq=1 Ack=1 Win=1 Len=0
8 5.026365900 192.168.1.5 20.33.105.0 TCP 54 [TCP ACKed unseen segment] 61352 → 443 [ACK] Seq=1 Ack=2 Win=255 Len=0
Frame 1: Packet, 1113 bytes on wire (8904 bits), 1113 bytes captured (8904 bits) on interface
Ethernet II, Src: NokiaShangha_15:66:20 (6c:22:f7:15:66:20), Dst: CloudNetwork_5a:18:e1 (3c:55
Internet Protocol Version 4, Src: 35.186.243.246, Dst: 192.168.1.5
Transmission Control Protocol, Src Port: 443, Dst Port: 53866, Seq: 1, Ack: 1, Len: 1059
Transport Layer Security
Wi-Fi: <live capture in progress>
Packets: 8
Profile: Default


Capturing from Wi-Fi (udp)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Lengt Info
96 7.046733800 192.168.1.5 172.217.24.174 QUIC 207 Protected Payload (KP0), DCID=f0f337f0d395a058
97 7.047496100 192.168.1.5 172.217.24.174 QUIC 81 Protected Payload (KP0), DCID=f0f337f0d395a058
98 7.047731700 192.168.1.5 172.217.24.174 QUIC 80 Protected Payload (KP0), DCID=f0f337f0d395a058
99 7.048700400 172.217.24.174 192.168.1.5 QUIC 73 Protected Payload (KP0)
100 7.060581600 172.217.24.174 192.168.1.5 QUIC 75 Protected Payload (KP0)
101 7.152502200 172.217.24.174 192.168.1.5 QUIC 309 Protected Payload (KP0)
102 7.152503800 172.217.24.174 192.168.1.5 QUIC 247 Protected Payload (KP0)
103 7.157443200 192.168.1.5 172.217.24.174 QUIC 77 Protected Payload (KP0), DCID=f0f337f0d395a058
104 7.216626400 172.217.24.174 192.168.1.5 QUIC 66 Protected Payload (KP0)
105 7.216979100 192.168.1.5 172.217.24.174 QUIC 72 Protected Payload (KP0), DCID=f0f337f0d395a058
106 7.228414700 172.217.24.174 192.168.1.5 QUIC 67 Protected Payload (KP0)
107 7.917863100 fe80::e94f:8d21:5a8… ff02::c UDP/XML 686 64014 → 3702 Len=624
108 8.190090600 fe80::e94f:8d21:5a8… ff02::c SSDP 157 M-SEARCH * HTTP/1.1
109 8.191765000 192.168.1.5 239.255.255.250 SSDP 143 M-SEARCH * HTTP/1.1
110 8.328458800 fe80::e94f:8d21:5a8… fe80::1 DNS 97 Standard query 0x4cd6 A crl3.digicert.com
Frame 1: Packet, 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface \Devic
Ethernet II, Src: CloudNetwork_5a:18:e1 (3c:55:76:5a:18:e1), Dst: NokiaShangha_15:66:20 (6c:22
Internet Protocol Version 6, Src: fe80::e94f:8d21:5a84:77ee, Dst: fe80::1
User Datagram Protocol, Src Port: 49689, Dst Port: 53
Domain Name System (query)
Wi-Fi: <live capture in progress>
Packets: 110
Profile: Default


The Wireshark Network Analyzer
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Welcome to Wireshark
4.6.8 (v4.6.8-0-ge677bf052328). You receive automatic updates.
Capture
...using this filter: Enter a capture filter ...
All interfaces shown
Wi-Fi
Ethernet 3
Local Area Connection* 7
Local Area Connection* 6
Local Area Connection* 5
Local Area Connection* 12
Local Area Connection* 10
Adapter for loopback traffic capture
Local Area Connection
Ethernet 2
Event Tracing for Windows (ETW) reader: etwdump
CONFERENCE
SharkFest'26 EU
Brussels, Belgium
November 2-6, 2026
Learn
Docs
Wiki
Q&A
Lists
Discord
Donate
Ready to load or capture
No Packets
Profile: Default

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits      : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code           : 0100011
Checking for errors...
   Error detected at bit position: 3 -> Correcting...
Corrected Code          : 0110011
Recovered Data Bits     : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 0
Original Data Bits      : 0110
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code           : 1100100
Checking for errors...
   Error detected at bit position: 6 -> Correcting...
Corrected Code          : 1100110
Recovered Data Bits     : 0110
```

```
=== SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (Go-Back-N) ===

Enter total number of frames to send     : 8
Enter sender's window size           : 4
Enter frame number to simulate as LOST
(0 to 7, enter -1 for no loss)         : 5

-----------------------------------------------
Sender Window : [Frame 0 - Frame 3]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node A --> Sending Frame 3
Node B --> Received Frame 0, sending ACK 0
Node B --> Received Frame 1, sending ACK 1
Node B --> Received Frame 2, sending ACK 2
Node B --> Received Frame 3, sending ACK 3
Node A --> ACK 3 received. Sliding window forward.

-----------------------------------------------
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
        (Frame 5 LOST in transit!)
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Frame 5 missing. Discarding out-of-order frames after it.
Node A --> Timeout! Resending window starting from Frame 5.

-----------------------------------------------
Sender Window : [Frame 4 - Frame 7]
Node A --> Sending Frame 5
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Received Frame 4, sending ACK 4
Node B --> Received Frame 5, sending ACK 5
Node B --> Received Frame 6, sending ACK 6
Node B --> Received Frame 7, sending ACK 7
Node A --> ACK 7 received. Sliding window forward.

-----------------------------------------------
All 8 frames successfully transmitted from Node A to Node B.
```

Send Packet

From:

computer1

To:

computer1

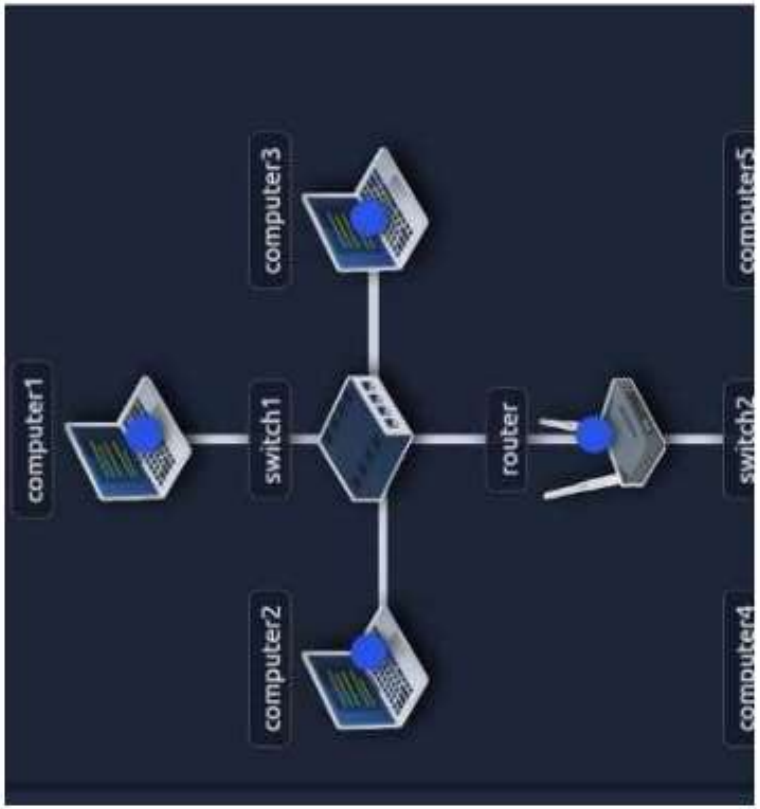
Packet Type:

arp_request

Data:

computer6

Send Packet



Send Packet

From:

computer4

To:

computer4

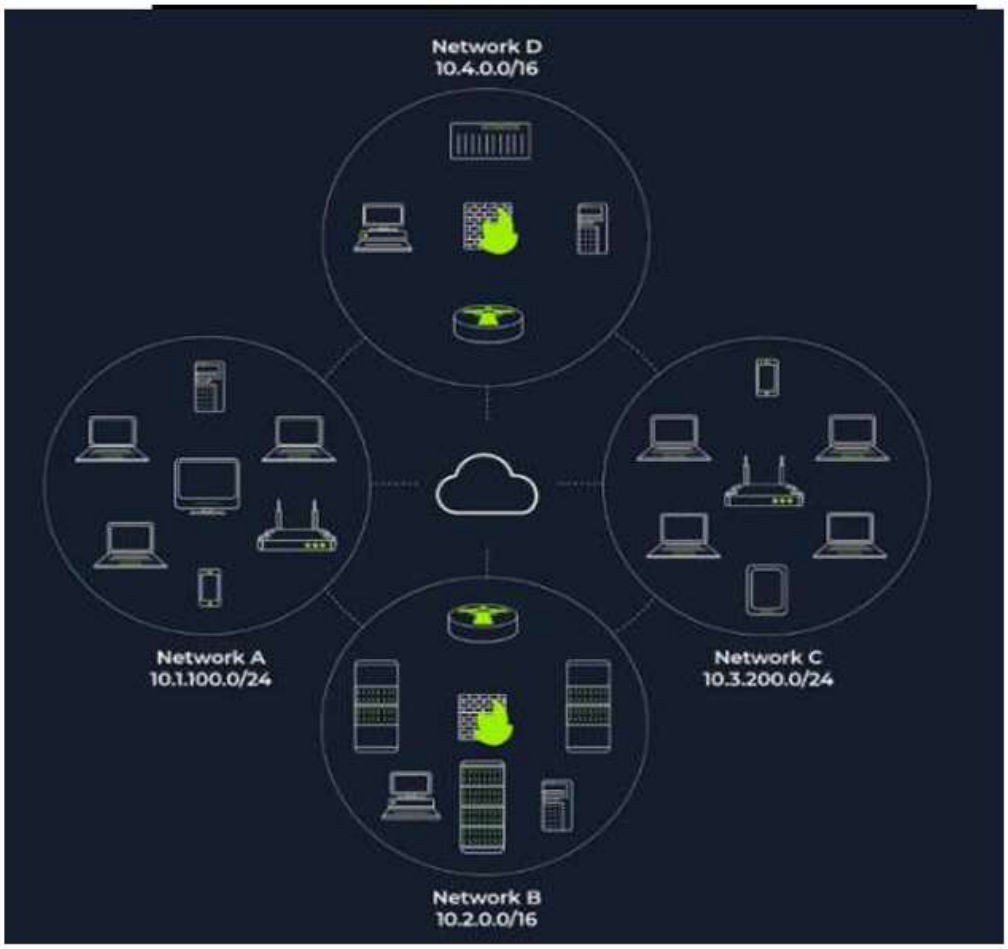
Packet Type:

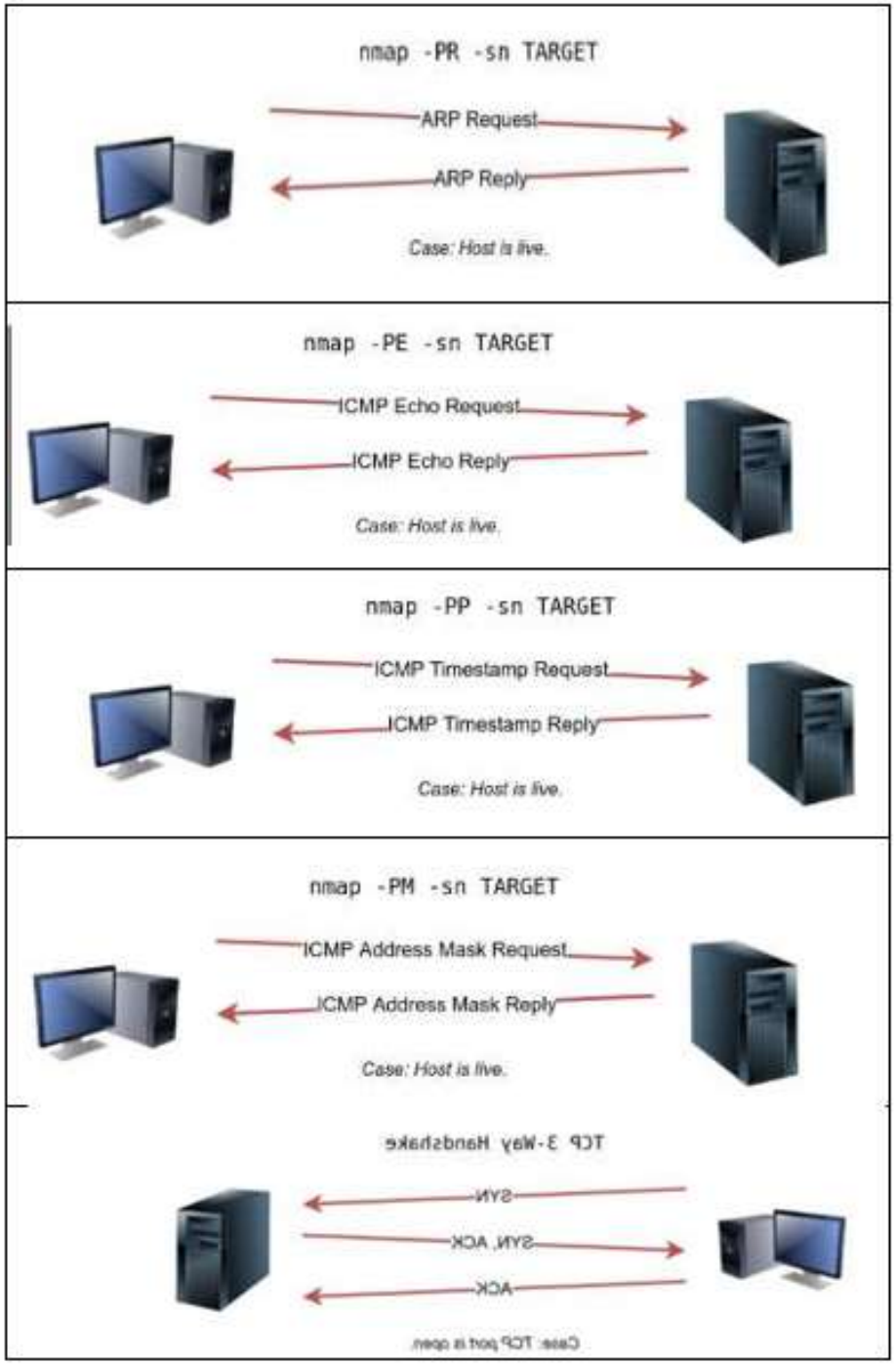
arp_request

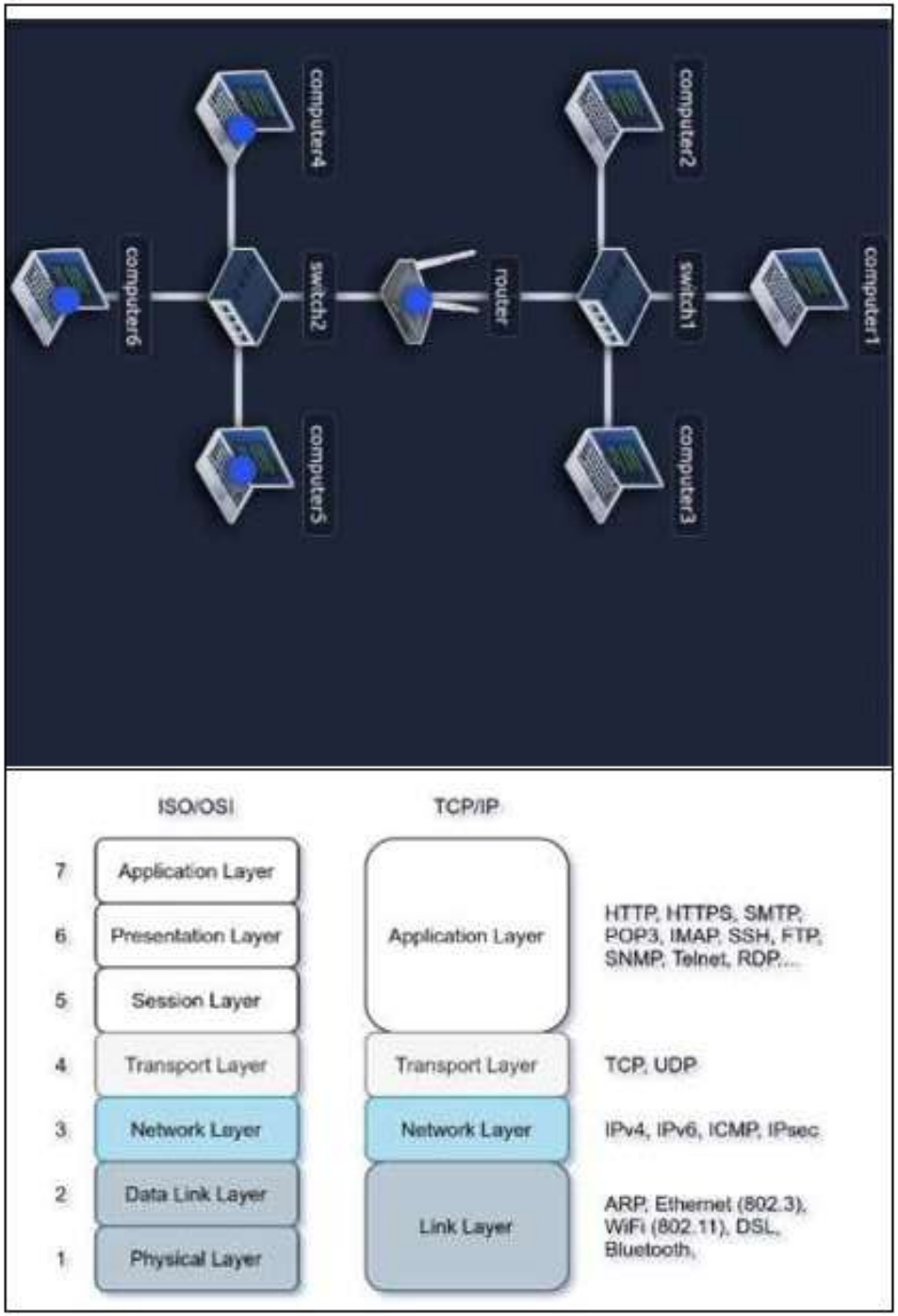
Data:

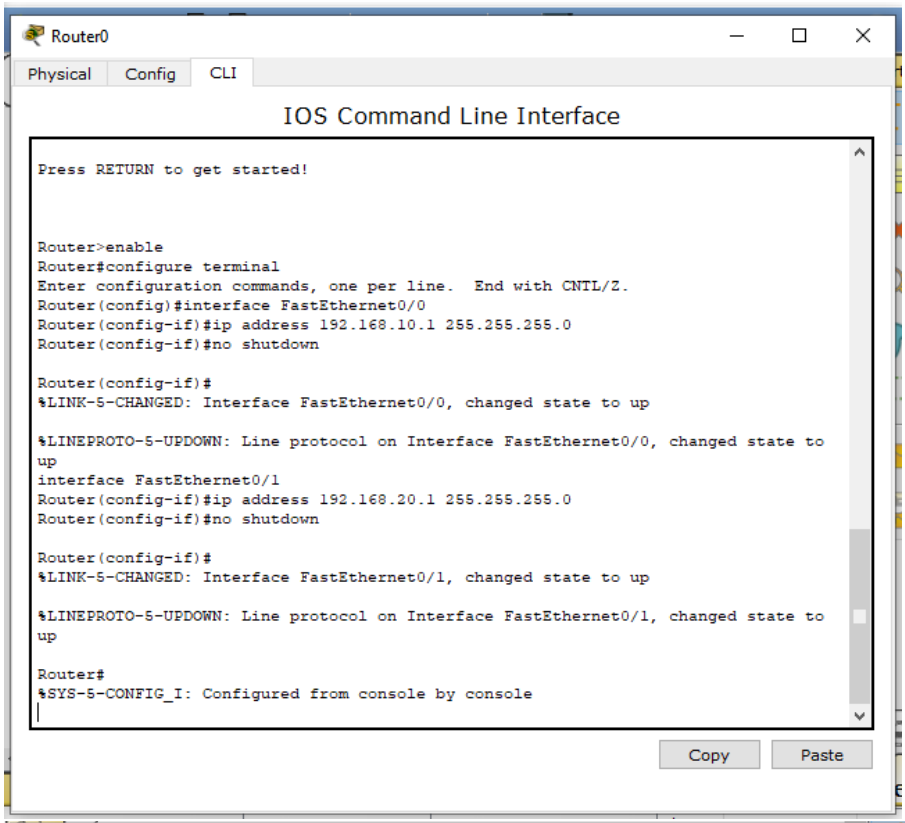
computer6

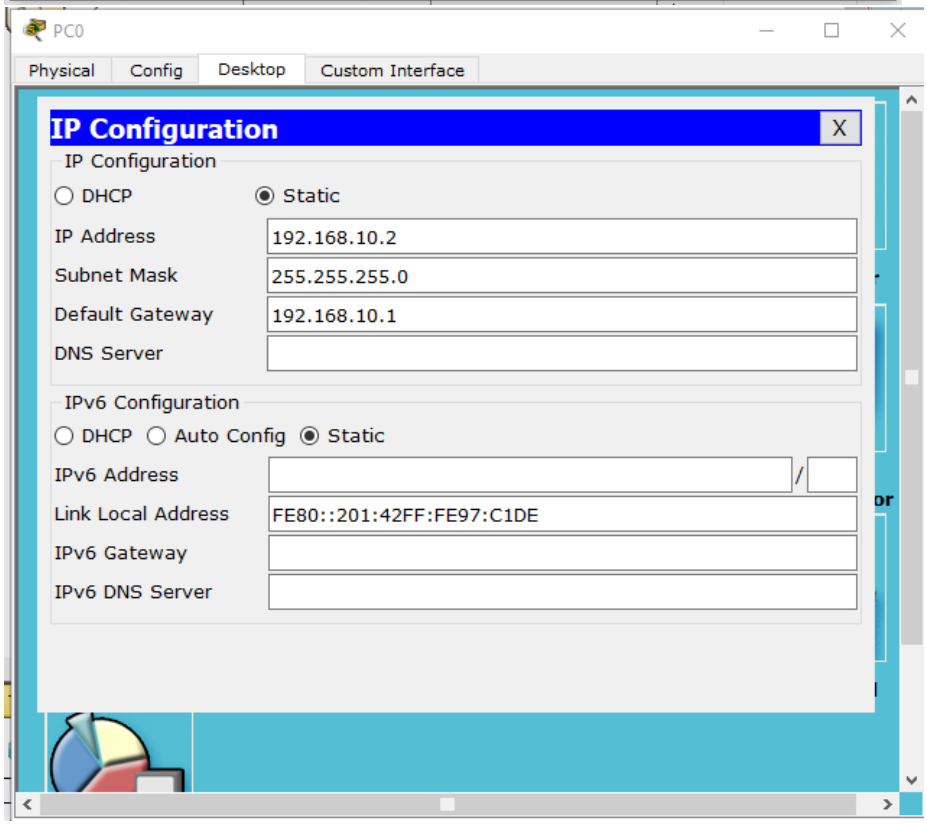
Send Packet

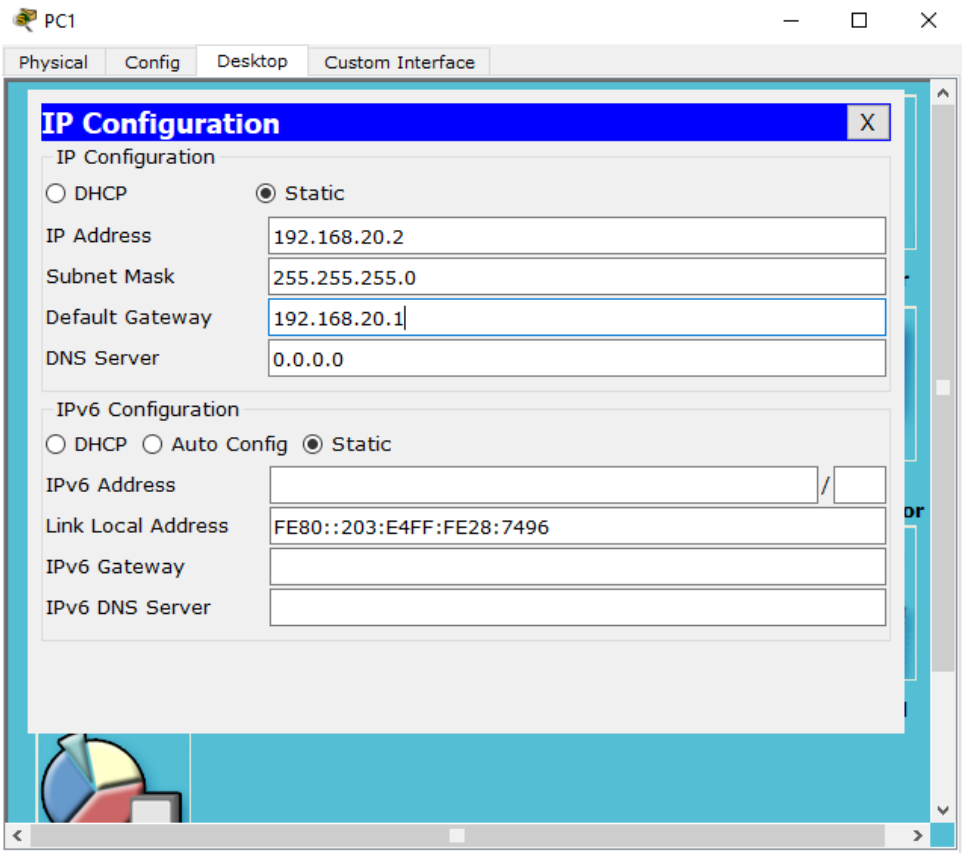

Network D
10.4.0.0/16
Network A
10.1.100.0/24
Network C
10.3.200.0/24
Network B
10.2.0.0/16

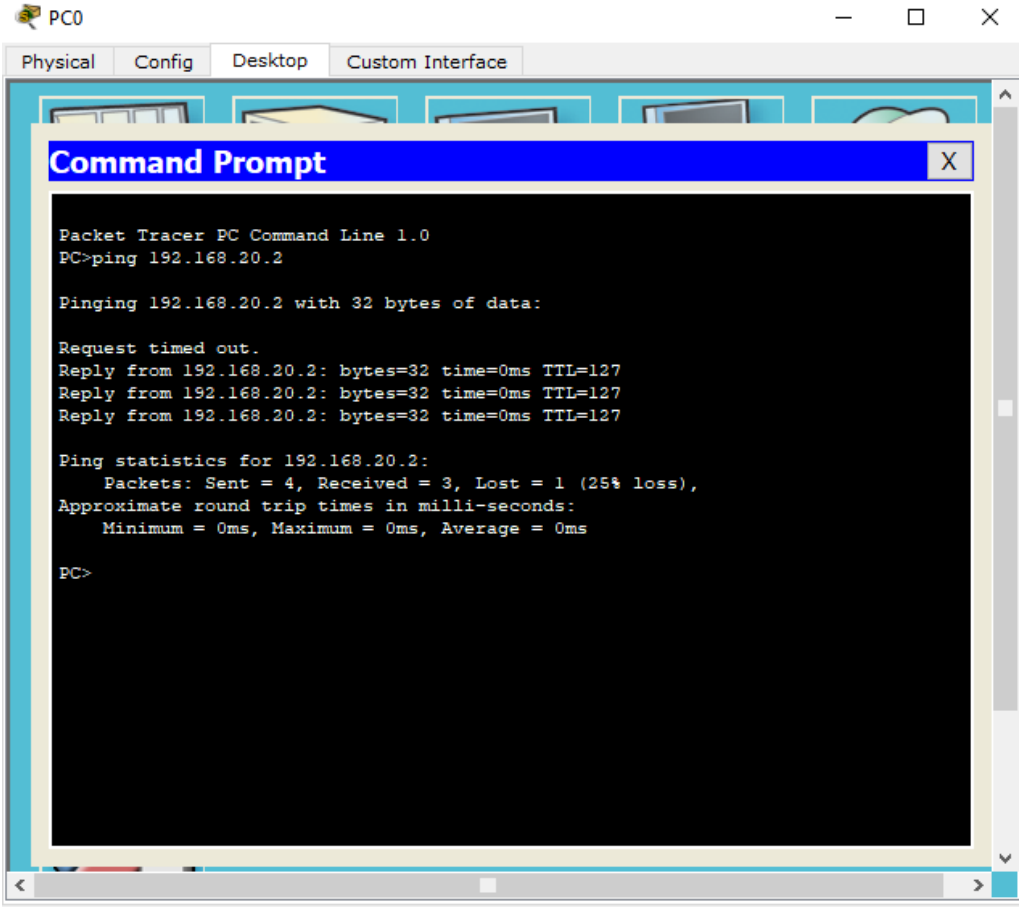

nmap -PR -sn TARGET
ARP Request
ARP Reply
Case: Host is live.
nmap -PE -sn TARGET
ICMP Echo Request
ICMP Echo Reply
Case: Host is live.
nmap -PP -sn TARGET
ICMP Timestamp Request
ICMP Timestamp Reply
Case: Host is live.
nmap -PM -sn TARGET
ICMP Address Mask Request
ICMP Address Mask Reply
Case: Host is live.

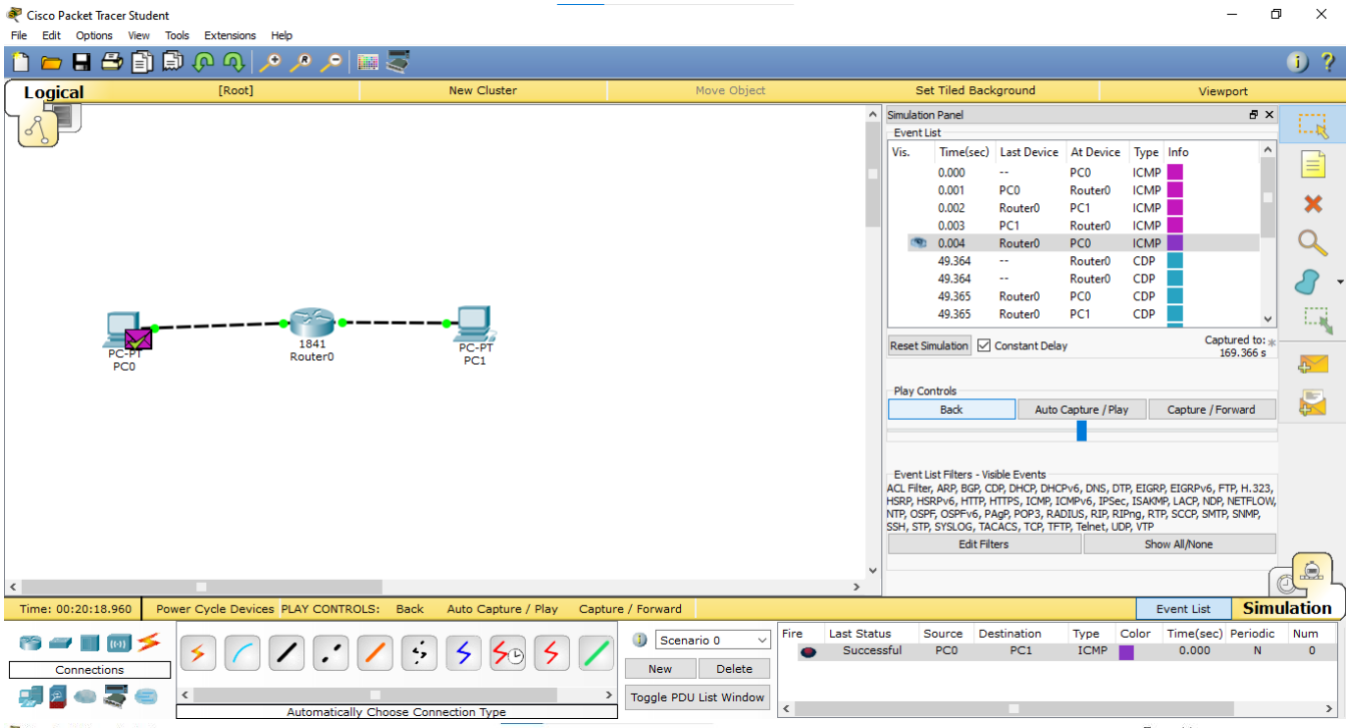

computer4
computer2
computer6
switch2
router
switch1
computer1
computer5
computer3
ISO/OSI
TCP/IP
7 Application Layer
6 Presentation Layer
5 Session Layer
4 Transport Layer
3 Network Layer
2 Data Link Layer
1 Physical Layer
Application Layer
Transport Layer
Network Layer
Link Layer
HTTP, HTTPS, SMTP, POP3, IMAP, SSH, FTP, SNMP, Telnet, RDP,...
TCP, UDP
IPv4, IPv6, ICMP, IPsec
ARP, Ethernet (802.3), WiFi (802.11), DSL, Bluetooth,

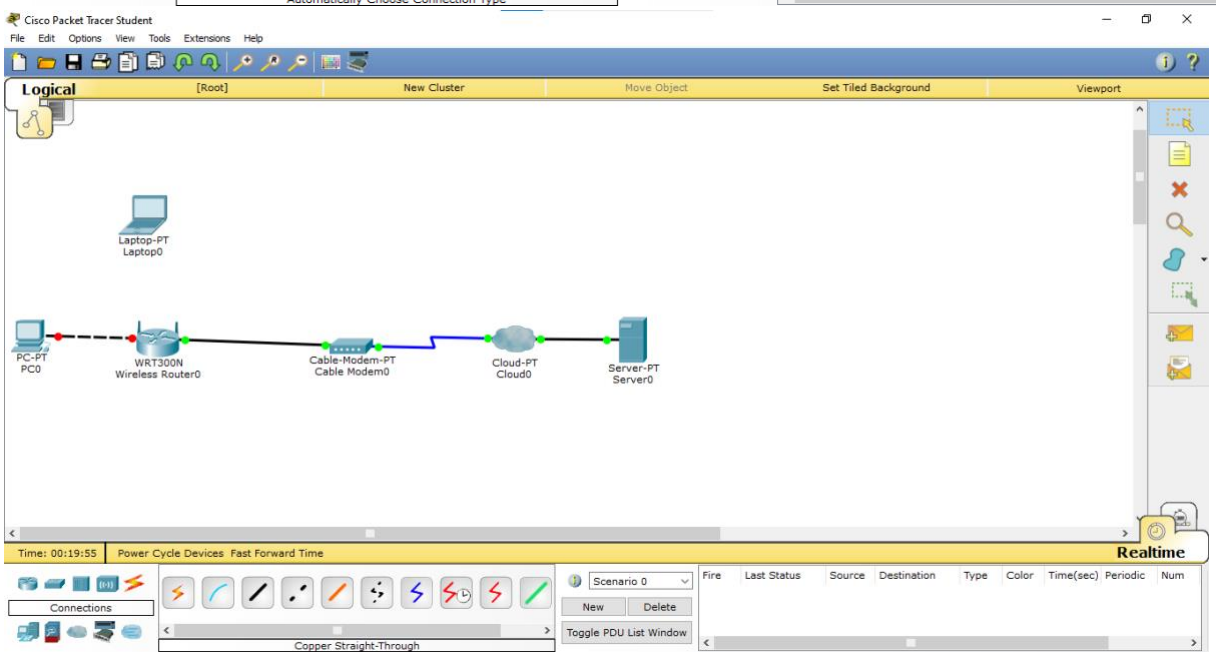

Router0
Physical
Config
CLI
IOS Command Line Interface
Press RETURN to get started!
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface FastEthernet0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
interface FastEthernet0/1
Router(config-if)#ip address 192.168.20.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up
Router#
%SYS-5-CONFIG_I: Configured from console by console
Copy
Paste

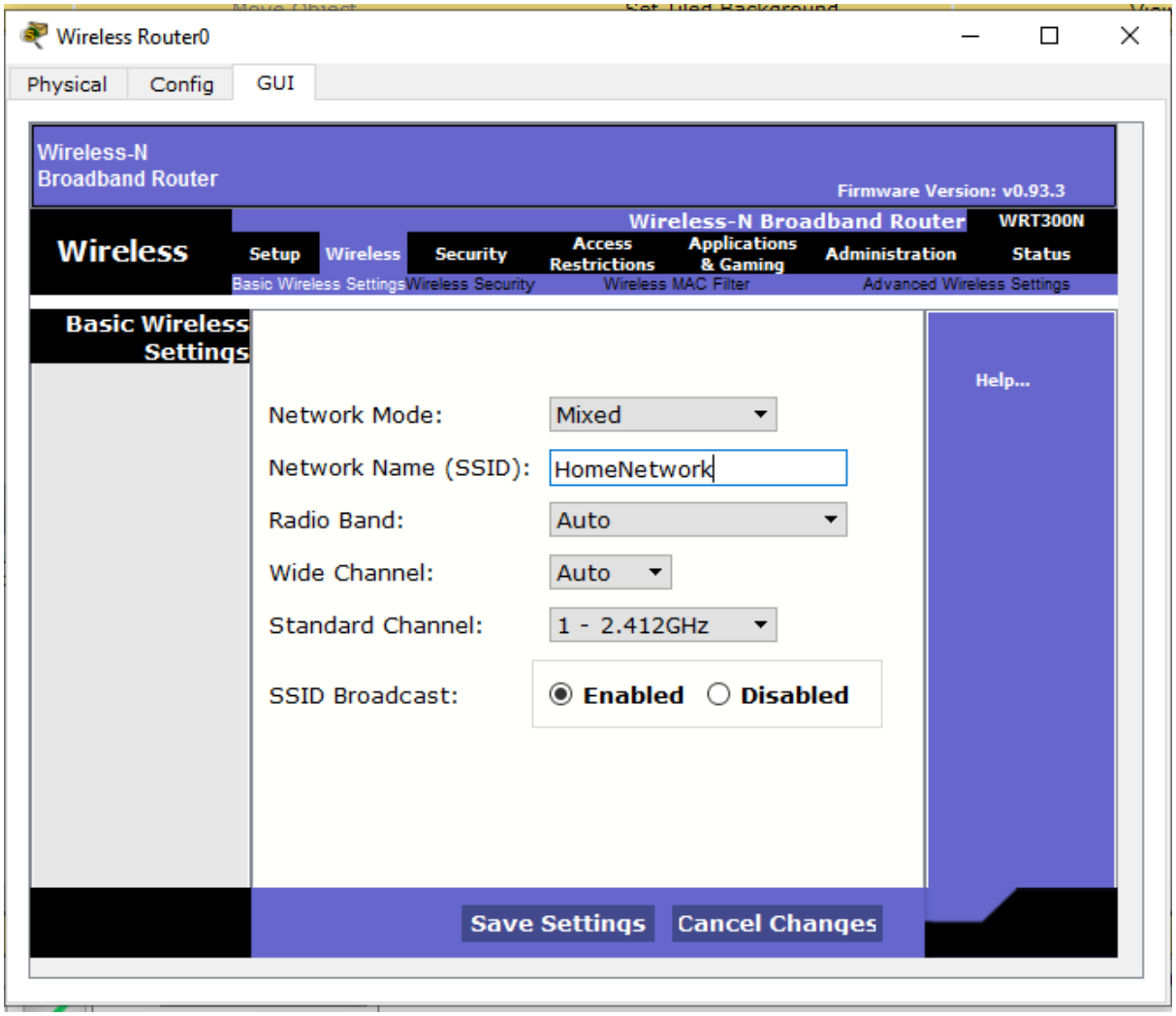

PC0
Physical
Config
Desktop
Custom Interface
IP Configuration
X
IP Configuration
DHCP
Static
IP Address
192.168.10.2
Subnet Mask
255.255.255.0
Default Gateway
192.168.10.1
DNS Server
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::201:42FF:FE97:C1DE
IPv6 Gateway
IPv6 DNS Server

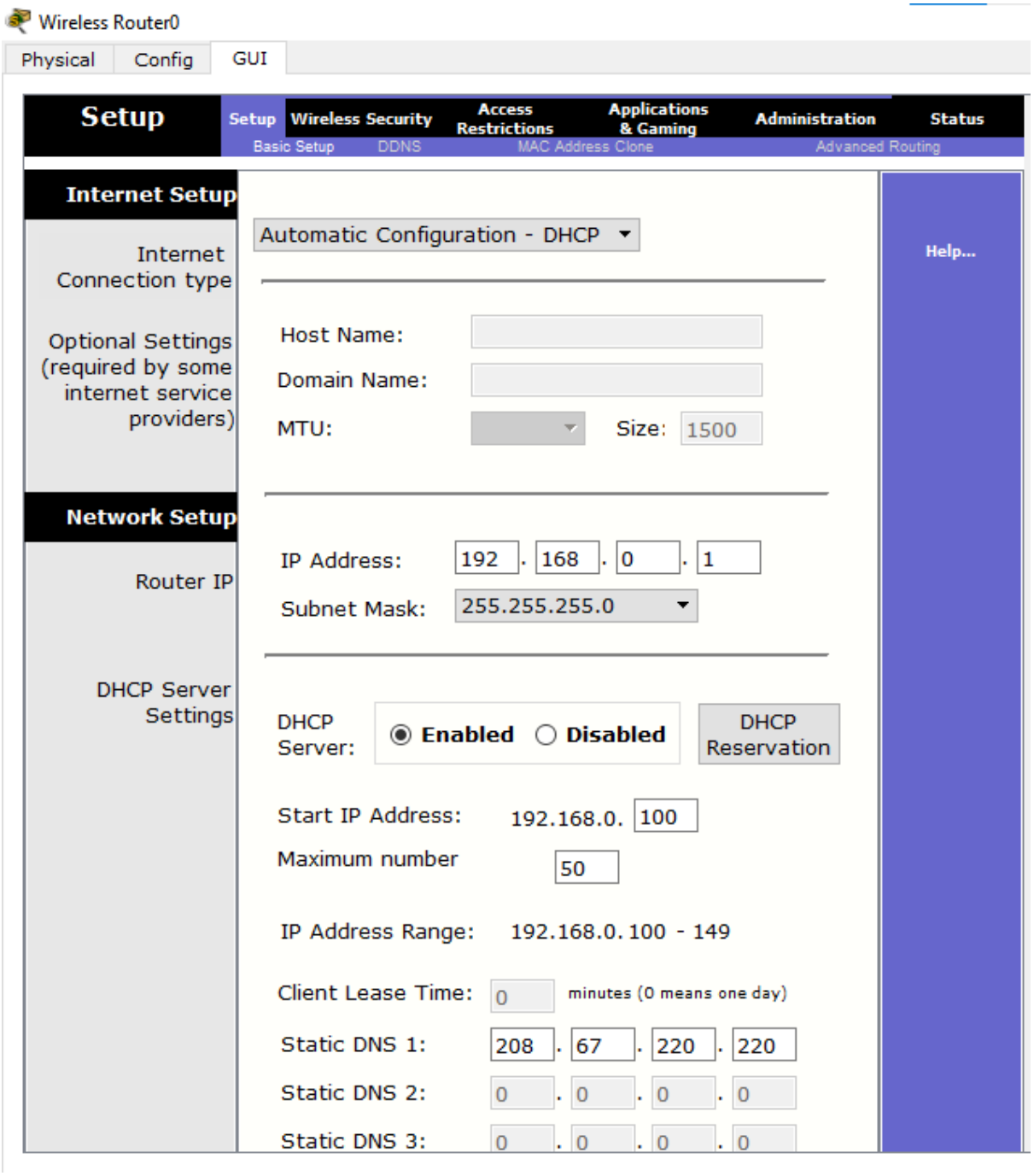

PC1
Physical
Config
Desktop
Custom Interface
IP Configuration
X
IP Configuration
DHCP
Static
IP Address
192.168.20.2
Subnet Mask
255.255.255.0
Default Gateway
192.168.20.1
DNS Server
0.0.0.0
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::203:E4FF:FE28:7496
IPv6 Gateway
IPv6 DNS Server


PC0
Physical
Config
Desktop
Custom Interface
Command Prompt
X
Packet Tracer PC Command Line 1.0
PC>ping 192.168.20.2
Pinging 192.168.20.2 with 32 bytes of data:
Request timed out.
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Ping statistics for 192.168.20.2:
Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms
PC>


Cisco Packet Tracer Student
File Edit Options View Tools Extensions Help
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
PC-PT PC0
1841 Router0
PC-PT PC1
Simulation Panel
Event List
Vis. Time(sec) Last Device At Device Type Info
0.000 -- PC0 ICMP
0.001 PC0 Router0 ICMP
0.002 Router0 PC1 ICMP
0.003 PC1 Router0 ICMP
0.004 Router0 PC0 ICMP
49.364 -- Router0 CDP
49.364 -- Router0 CDP
49.365 Router0 PC0 CDP
49.365 Router0 PC1 CDP
Reset Simulation
Constant Delay
Captured to: 169.366 s
Play Controls
Back
Auto Capture / Play
Capture / Forward
Event List Filters - Visible Events
ACL Filter, ARP, BGP, CDP, DHCP, DHCPv6, DNS, DTP, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IPSec, ISAKMP, LACP, NDP, NETFLOW, NTP, OSPF, OSPFv6, PAgP, POP3, RADIUS, RIP, RIPng, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP, VTP
Edit Filters
Show All/None
Time: 00:20:18.960
Power Cycle Devices PLAY CONTROLS: Back Auto Capture / Play Capture / Forward
Event List
Simulation
Connections
Automatically Choose Connection Type
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num
Successful PC0 PC1 ICMP 0.000 N 0


Cisco Packet Tracer Student
File Edit Options View Tools Extensions Help
Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
Laptop-PT Laptop0
PC-PT PC0
WRT300N Wireless Router0
Cable-Modem-PT Cable Modem0
Cloud-PT Cloud0
Server-PT Server0
Time: 00:19:55
Power Cycle Devices Fast Forward Time
Realtime
Connections
Copper Straight-Through
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num


Wireless Router0
Physical
Config
GUI
Wireless-N
Broadband Router
Firmware Version: v0.93.3
Wireless-N Broadband Router
WRT300N
Wireless
Setup
Wireless
Security
Access Restrictions
Applications & Gaming
Administration
Status
Basic Wireless Settings
Wireless Security
Wireless MAC Filter
Advanced Wireless Settings
Basic Wireless Settings
Help...
Network Mode:
Mixed
Network Name (SSID):
HomeNetwork
Radio Band:
Auto
Wide Channel:
Auto
Standard Channel:
1 - 2.412GHz
SSID Broadcast:
Enabled
Disabled
Save Settings
Cancel Changes


Wireless Router0
Physical
Config
GUI
Setup
Setup
Wireless Security
Access Restrictions
Applications & Gaming
Administration
Status
Basic Setup
DDNS
MAC Address Clone
Advanced Routing
Internet Setup
Internet Connection type
Automatic Configuration - DHCP
Optional Settings (required by some internet service providers)
Host Name:
Domain Name:
MTU:
Size:
1500
Network Setup
Router IP
IP Address:
192
168
0
1
Subnet Mask:
255.255.255.0
DHCP Server Settings
DHCP Server:
Enabled
Disabled
DHCP Reservation
Start IP Address:
192.168.0.
100
Maximum number
50
IP Address Range:
192.168.0.100 - 149
Client Lease Time:
0
minutes (0 means one day)
Static DNS 1:
208
67
220
220
Static DNS 2:
Static DNS 3:
Help...

EXP 14

AIM

To write and implement a network packet sniffer using Raw Sockets in C to capture and parse IP, TCP, and UDP network headers.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Open a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_TCP (or ETH_P_ALL for lower-level link layer capture).
2. **Buffer Allocation:** Allocate a buffer (e.g., 65536 bytes) to hold incoming network frames and packets.
3. **Capture Loop:**
    - Call recvfrom() to continuously capture network packets passing through the interface.
4. **Parse Headers:**
    - **Extract IP Header:** Cast the buffer pointer to an struct iphdr pointer. Extract fields like Version, Header Length, TTL, Protocol, Source IP, and Destination IP.
    - **Identify Protocol:** Check the protocol field (ip_hdr->protocol).
    - **Extract Transport Header:**
        - If Protocol is 6 (TCP): Cast buffer offset to struct tcphdr and extract Source Port, Destination Port, and Flags (SYN, ACK, FIN, etc.).

      If Protocol is 17 (UDP): Cast buffer offset to struct udphdr and extract Source Port, Destination Port, and Length.
5. **Display Packet Info:** Print formatted packet metadata to the console.
6. **Clean Up:** Close the socket upon receiving a termination signal.

Code Implementation (packet_sniffer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>
#include <netinet/udp.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 65536

void process_packet(unsigned char *buffer, int size) {
    // Extract IP Header
    struct iphdr *ip_hdr = (struct iphdr *)buffer;
    unsigned int ip_hdr_len = ip_hdr->ihl * 4;

    struct sockaddr_in source, dest;
    memset(&source, 0, sizeof(source));
    source.sin_addr.s_addr = ip_hdr->saddr;

    memset(&dest, 0, sizeof(dest));
    dest.sin_addr.s_addr = ip_hdr->daddr;

    printf("\n================ Packet Captured
================\n");
```

```
printf("IP Header:\n");
printf("   |-IP Version        : %d\n", (unsigned int)ip_hdr->version);
printf("   |-Header Length     : %d Bytes\n", ip_hdr_len);
printf("   |-Time To Live (TTL): %d\n", ip_hdr->ttl);
printf("   |-Protocol          : %d\n", ip_hdr->protocol);
printf("   |-Source IP         : %s\n", inet_ntoa(source.sin_addr));
printf("   |-Destination IP    : %s\n", inet_ntoa(dest.sin_addr));

// Handle TCP Packets
if (ip_hdr->protocol == IPPROTO_TCP) {
    struct tcphdr *tcp_hdr = (struct tcphdr *)(buffer + ip_hdr_len);
    printf("TCP Header:\n");
    printf("   |-Source Port       : %u\n", ntohs(tcp_hdr->source));
    printf("   |-Destination Port  : %u\n", ntohs(tcp_hdr->dest));
    printf("   |-Sequence Number   : %u\n", ntohl(tcp_hdr->seq));
    printf("   |-Acknowledge No    : %u\n", ntohl(tcp_hdr->ack_seq));
    printf("   |-SYN Flag          : %d\n", tcp_hdr->syn);
    printf("   |-ACK Flag          : %d\n", tcp_hdr->ack);
    printf("   |-FIN Flag          : %d\n", tcp_hdr->fin);
}
// Handle UDP Packets
else if (ip_hdr->protocol == IPPROTO_UDP) {
    struct udphdr *udp_hdr = (struct udphdr *)(buffer + ip_hdr_len);
    printf("UDP Header:\n");
    printf("   |-Source Port       : %u\n", ntohs(udp_hdr->source));
    printf("   |-Destination Port  : %u\n", ntohs(udp_hdr->dest));
```

```
        printf("   |-UDP Length        : %u\n", ntohs(udp_hdr->len));
    }

printf("============================================
=\n");
}

int main() {
    int raw_sock;
    socklen_t saddr_len;
    struct sockaddr saddr;
    unsigned char *buffer = (unsigned char *)malloc(BUFFER_SIZE);

    if (buffer == NULL) {
        printf("Buffer allocation failed.\n");
        return 1;
    }

    // Create raw socket to capture IP packets
    raw_sock = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
    if (raw_sock < 0) {
        perror("Socket creation error (ensure you run with sudo)");
        free(buffer);
        return 1;
    }

    printf("Starting Packet Sniffer...\nPress Ctrl+C to stop.\n");
```

```
    // Capture 3 packets as an example output
    int packet_count = 0;
    while (packet_count < 3) {
        saddr_len = sizeof(saddr);
        int data_size = recvfrom(raw_sock, buffer, BUFFER_SIZE, 0,
&saddr, &saddr_len);

        if (data_size < 0) {
            perror("Recvfrom error");
            free(buffer);
            close(raw_sock);
            return 1;
        }

        process_packet(buffer, data_size);
        packet_count++;
    }

    close(raw_sock);
    free(buffer);
    return 0;
}
```

Input / Output

Input

Command Terminal:

sudo ./sniffer

Output

Terminal Screen:

Starting Packet Sniffer...

Press Ctrl+C to stop.

================ Packet Captured ================

IP Header:

|-IP Version : 4

|-Header Length : 20 Bytes

|-Time To Live (TTL): 64

|-Protocol : 6

|-Source IP : 192.168.1.5

|-Destination IP : 93.184.216.34

TCP Header:

|-Source Port : 49152

|-Destination Port : 80

|-Sequence Number : 10423011

|-Acknowledge No : 0

|-SYN Flag : 1

|-ACK Flag : 0

|-FIN Flag : 0

==

EXP 15

AIM

To analyze different types of web logs using the Webalizer log analysis tool and generate web traffic reports.

Procedure

Step 1: Run Webalizer

1. Download and extract the Webalizer executable (webalizer.exe) on Windows.
2. Place the executable in a dedicated working folder (e.g., C:\webalizer).

Step 2: Input Web Log File

1. Obtain or download a sample web server access log file (e.g., access.log from Apache, Nginx, or IIS).
2. Move the log file into the same directory as webalizer.exe or update the webalizer.conf configuration file to point to the input log location:

LogFile access.log

OutputDir ./output

Step 3: Run Webalizer

1. Open Command Prompt (cmd) and navigate to the Webalizer directory:

DOS

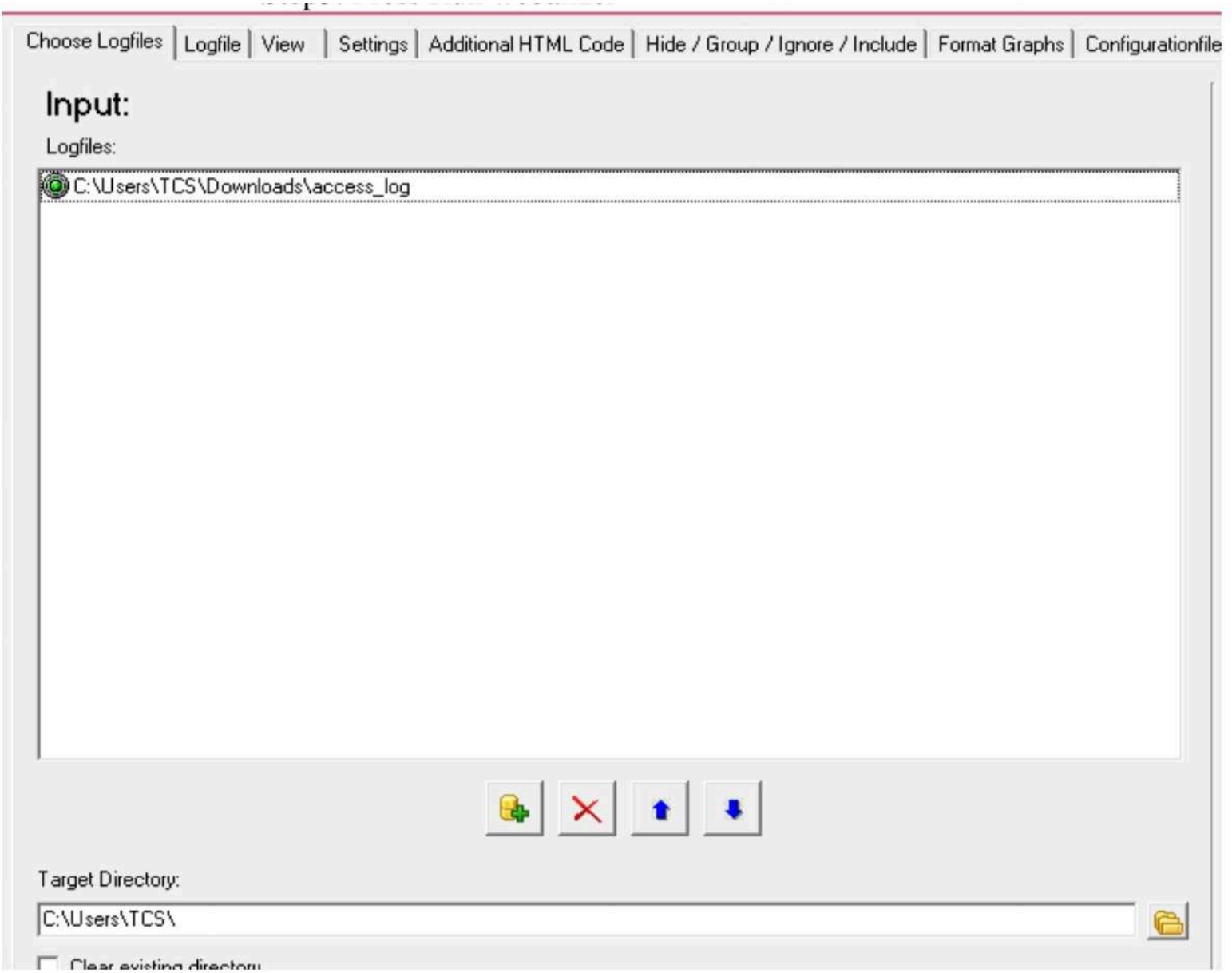
cd C:\webalizer

2. Execute Webalizer specifying the log file:

DOS

webalizer.exe access.log

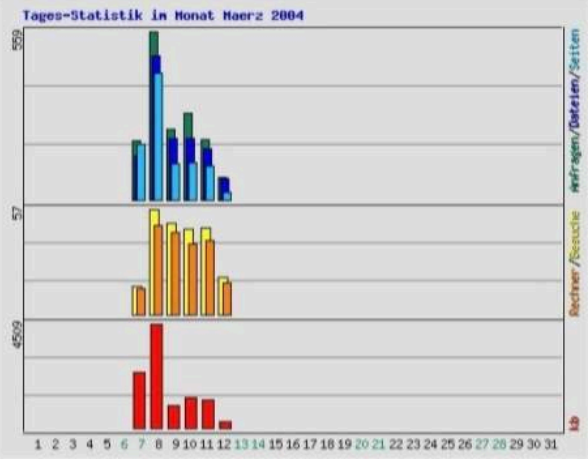
3. Open the generated index.html file in any web browser to view the generated traffic report.


Choose Logfiles
Logfile
View
Settings
Additional HTML Code
Hide / Group / Ignore / Include
Format Graphs
Configurationfile
Input:
Logfiles:
C:\Users\TCS\Downloads\access_log
Target Directory:
C:\Users\TCS\

Monthly statistics

March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries



Seiten - Dokument-Abfragen (z.B. html, asp, txt); Dateien - erfolgreiche Anfragen für alle Typen (z.B. html, gif, css); Treffer - alle Anfragen, inklusive Fehler;

Tages-Statistik im Monat Maerz 2004

Tag	Anfragen			Dateien			Seiten			Besuche			Rechner			kb		
	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum
7	197 12.74%	25	29	147 11.54%	18	21	186 18.94%	23	27	15 6.41%	2	3	14 8.19%	2	3	2383 22.27%	298	542
8	559 36.16%	24	63	478 37.52%	21	56	419 42.67%	18	40	57 24.36%	2	7	48 28.07%	2	7	4509 42.14%	196	386
9	232 15.01%	11	29	205 16.09%	10	29	118 12.02%	6	16	49 20.94%	2	5	44 25.73%	2	5	955 8.93%	45	138
10	286 18.50%	14	102	205 16.09%	10	34	124 12.63%	6	28	46 19.66%	2	5	38 22.22%	2	4	1336 12.49%	67	379

Hosts

March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries

Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	100 6.47%	83 6.51%	46 4.68%	472 4.41%	10 4.27%	8.48	39.85	shawcable.net
2	72 4.66%	71 5.57%	52 5.30%	86 0.80%	71 30.34%	0.09	6.62	inktomisearch.com
3	47 3.04%	41 3.22%	43 4.38%	613 5.73%	4 1.71%	3.77	15.10	overture.com
4	44 2.85%	42 3.30%	23 2.34%	244 2.28%	2 0.85%	4.51	4.70	hevanet.com
5	35 2.26%	29 2.28%	14 1.43%	218 2.03%	7 2.99%	1.31	6.00	bc.ca
6	29 1.88%	28 2.20%	14 1.43%	97 0.91%	1 0.43%	3.43	3.43	panduit.com
7	23 1.49%	14 1.10%	10 1.02%	135 1.26%	1 0.43%	12.35	12.35	geovariances.fr
8	22 1.42%	22 1.73%	8 0.81%	67 0.63%	1 0.43%	3.72	3.72	cox.net
9	19 1.23%	19 1.49%	7 0.71%	61 0.57%	1 0.43%	1.52	1.52	ac.il
10	19 1.23%	11 0.86%	10 1.02%	51 0.48%	2 0.85%	3.04	5.87	netinfo.bg
11	15 0.97%	14 1.10%	13 1.32%	130 1.22%	3 1.28%	1.29	3.87	e-i.net
12	13 0.84%	13 1.02%	13 1.32%	120 1.12%	1 0.43%	3.88	3.88	telia.net
13	13 0.84%	13 1.02%	1 0.10%	28 0.26%	1 0.43%	0.13	0.13	net.ar
14	13 0.84%	13 1.02%	2 0.20%	30 0.28%	1 0.43%	0.87	0.87	dhl.com
15	12 0.78%	11 0.86%	9 0.92%	68 0.63%	1 0.43%	25.53	25.53	tiscali.de
16	12 0.78%	12 0.94%	1 0.10%	27 0.25%	1 0.43%	0.33	0.33	wtbts.org
17	12 0.78%	12 0.94%	1 0.10%	27 0.25%	1 0.43%	0.10	0.10	qwest.net
18	11 0.71%	11 0.86%	7 0.71%	41 0.38%	2 0.85%	0.56	1.03	radiant.net
19	10 0.65%	9 0.71%	7 0.71%	41 0.38%	1 0.43%	1.40	1.40	net.au
20	10 0.65%	10 0.78%	2 0.20%	61 0.57%	1 0.43%	0.17	0.17	3_343_lt_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	47 3.04%	41 3.22%	43 4.38%	613 5.73%	4 1.71%	3.77	15.10	overture.com
2	100 6.47%	83 6.51%	46 4.68%	472 4.41%	10 4.27%	8.48	39.85	shawcable.net

User-agents

March 2004 | Last 12 Months | Monthly Statistics | Daily Statistics | Hourly Statistics | URLs

Entry Pages | Exit Pages | Errors | Hosts | Referrer | Searches | Usernames | Useragents | Countries

9	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62	**inktomisearch.com**
10	4	0.26%	4	0.31%	4	0.41%	74	0.69%	4	1.71%	0.00	0.00	**rhyolite.com**

Top 1 von 1 Verweise

#	Anfragen		Besuche		Verweise
1	1546	100.00%	234	100.00%	- (Direkte Anfrage)

Top 1 von 1 Anwenderprogramme

#	Anfragen		kb		Besuche		Anwenderprogramm
1	1546	100.00%	10699	100.00%	234	100.00%	-



Top 1 von 1 Ländern

#	Anfragen		Dateien		kb		Besuche		Land
1	1543	99.81%	1271	99.76%	10684	99.85%	231	100.00%	Unbekannte Adressen

100% Micro$oft free!

AIM

To implement a custom Ping program using raw ICMP (Internet Control Message Protocol) sockets in C.

Algorithm

Main Program Flow

1. **Create Raw Socket:** Initialize a socket using domain AF_INET, type SOCK_RAW, and protocol IPPROTO_ICMP.
2. **Resolve Host IP:** Convert the domain name or IPv4 string into a valid target IP address using DNS lookup.
3. **Construct ICMP Packet:**
    - Set ICMP type to ICMP_ECHO (Type 8, Code 0).
    - Set Sequence Number and Process Identifier (PID) fields.
    - Calculate and set the Internet Checksum over the entire ICMP header and payload.
4. **Send Echo Request:** Transmit the packet to the destination address using sendto(). Record the transmission timestamp.
5. **Receive Echo Reply:**
    - Wait for incoming packets with recvfrom() within a timeout window.
    - Parse the outer IP header to extract the inner ICMP packet.

    - Verify that the received packet is an ICMP_ECHOREPLY (Type 0) matching the process ID and sequence number.
6. **Calculate Metrics:** Subtract transmission timestamp from receive timestamp to obtain Round Trip Time (RTT).
7. **Display Output:** Print sequence number, TTL (Time to Live), packet size, and RTT. Repeat for desired number of pings.

Code Implementation (ping.c)

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/socket.h>

#include <netinet/in.h>

#include <netinet/ip.h>

#include <netinet/ip_icmp.h>

#include <arpa/inet.h>

#include <netdb.h>

#include <sys/time.h>


#define PACKET_SIZE 64

#define PORT_NO 0

#define TIMEOUT 1 // Timeout in seconds
```

```
// Standard Internet Checksum Algorithm
unsigned short calculate_checksum(unsigned short *buffer, int length) {
    unsigned long sum = 0;
    while (length > 1) {
        sum += *buffer++;
        length -= 2;
    }
    if (length == 1) {
        sum += *(unsigned char *)buffer;
    }
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    return (unsigned short)(~sum);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: sudo %s <hostname/IP>\n", argv[0]);
        return 1;
    }

    int sockfd;
```

```
    struct sockaddr_in dest_addr;
    struct hostent *host;
    char packet[PACKET_SIZE];
    struct icmp *icmp_hdr;
    struct timeval timeout, tv_start, tv_end;

    // 1. Resolve Hostname
    host = gethostbyname(argv[1]);
    if (host == NULL) {
        perror("Could not resolve hostname");
        return 1;
    }

    memset(&dest_addr, 0, sizeof(dest_addr));
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_addr = *((struct in_addr *)host-
>h_addr_list[0]);

    // 2. Create Raw Socket for ICMP
    sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
    if (sockfd < 0) {
        perror("Socket creation failed (run as root)");
        return 1;
    }
```

```
    // Set Receive Timeout
    timeout.tv_sec = TIMEOUT;
    timeout.tv_usec = 0;
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, (const
char*)&timeout, sizeof(timeout));


    printf("PING %s (%s) %d bytes of data.\n", argv[1],
inet_ntoa(dest_addr.sin_addr), PACKET_SIZE);


    for (int seq = 1; seq <= 4; seq++) {
        memset(packet, 0, PACKET_SIZE);


        // 3. Construct ICMP Header
        icmp_hdr = (struct icmp *)packet;
        icmp_hdr->icmp_type = ICMP_ECHO;
        icmp_hdr->icmp_code = 0;
        icmp_hdr->icmp_id = getpid();
        icmp_hdr->icmp_seq = seq;
        icmp_hdr->icmp_cksum = 0;


        // Fill remaining bytes with sample data payload
        for (int i = sizeof(struct icmp); i < PACKET_SIZE; i++) {
            packet[i] = i + '0';
```

```
        }

        icmp_hdr->icmp_cksum = calculate_checksum((unsigned
short *)packet, PACKET_SIZE);

        // 4. Send Packet & Record Time
        gettimeofday(&tv_start, NULL);
        if (sendto(sockfd, packet, PACKET_SIZE, 0, (struct
sockaddr *)&dest_addr, sizeof(dest_addr)) <= 0) {
            perror("Failed to send packet");
            continue;
        }

        // 5. Receive Reply
        char recv_buf[1024];
        struct sockaddr_in from_addr;
        socklen_t from_len = sizeof(from_addr);

        int bytes_rcvd = recvfrom(sockfd, recv_buf,
sizeof(recv_buf), 0, (struct sockaddr *)&from_addr, &from_len);
        gettimeofday(&tv_end, NULL);

        if (bytes_rcvd < 0) {
            printf("Request timed out for icmp_seq %d\n", seq);
        } else {
```

```
            // Extract IP header to inspect TTL and payload
            struct ip *ip_hdr = (struct ip *)recv_buf;
            int ip_hdr_len = ip_hdr->ip_hl * 4;
            struct icmp *reply_icmp = (struct icmp *)(recv_buf +
ip_hdr_len);

            // Calculate RTT in milliseconds
            double rtt = (tv_end.tv_sec - tv_start.tv_sec) * 1000.0 +
                         (tv_end.tv_usec - tv_start.tv_usec) / 1000.0;

            if (reply_icmp->icmp_type == ICMP_ECHOREPLY) {
                printf("%d bytes from %s: icmp_seq=%d ttl=%d
time=%.2f ms\n",
                       bytes_rcvd - ip_hdr_len,
                       inet_ntoa(from_addr.sin_addr),
                       reply_icmp->icmp_seq,
                       ip_hdr->ip_ttl,
                       rtt);
            }
        }
        sleep(1); // Wait 1 second between requests
    }

    close(sockfd);
```

```
    return 0;
}
```

Input / Output

Input

Command Terminal:

sudo ./myping 127.0.0.1

Output

Server Terminal:

PING 127.0.0.1 (127.0.0.1) 64 bytes of data.

64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.08 ms

64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.06 ms

64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.07 ms

64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.05 ms